

**Register Number:192425325**

**Name:B.Archana**

**CO2-AT-3**

**MLA0201-Fundamentals Of Machine Learning**

**Debugging & Optimisation Task**

**Q1. A real estate company developed a linear regression model to predict house prices, but the model is showing high prediction error.**

- (i) Identify possible causes for the high error (e.g., multicollinearity, outliers, poor feature selection).**
- (ii) Analyze the dataset/model behavior to detect these issues.**
- (iii) Apply suitable optimization techniques (such as feature scaling, removing outliers, or adding relevant features) to improve the model performance.**

### **Debugging and Optimizing a Linear Regression Model for House Price Prediction**

A real estate company uses Linear Regression to predict house prices, but the model produces a high prediction error. This may happen because the dataset contains highly correlated features, outliers, irrelevant variables, missing values, or features that are not properly scaled. The problem can be analyzed systematically and the model can then be optimized.

#### **(i) Possible Causes of High Prediction Error:**

##### **1. Multicollinearity**

**Multicollinearity** occurs when two or more independent variables are highly correlated with each other.

For example:

- GarageCars and GarageArea
- TotalBsmtSF and basement-related area variables
- 1stFlrSF and TotalBsmtSF
- GrLivArea and TotRmsAbvGrd

When highly correlated variables are given to Linear Regression, the model may have difficulty determining the individual contribution of each variable. This can result in unstable coefficients and poor generalization.

**Detection:**

Correlation matrix and **Variance Inflation Factor (VIF)** can be used.

**2. Outliers**

An **outlier** is a data point that is significantly different from the majority of observations.

For example, a very large house with extremely high GrLivArea may have a disproportionately high SalePrice.

Outliers can strongly affect the regression line because Linear Regression tries to minimize squared errors.

**Detection methods:**

- Box plots
- IQR method
- Scatter plots
- Standardized residuals
- Cook's distance

**3. Poor Feature Selection**

Using irrelevant or weakly related features can increase model complexity without improving prediction.

For example, some variables may contribute very little to house price while important variables such as:

- Overall quality
- Living area
- Garage capacity
- Number of bathrooms
- Year built
- Neighborhood

can have a much stronger relationship with price.

Therefore, feature selection should be performed.

**4. Missing Values**

Real-world datasets frequently contain missing values.

If missing values are not handled properly, the model may:

- fail during training,
- lose many observations,
- or learn biased relationships.

The Kaggle dataset contains several columns with missing values, and some missing categorical values actually indicate that a property does not have that feature—for example, no fence or no garage.

## **(ii) Analyze the Dataset/Model to Detect the Problems:**

The following analysis can be performed.

### **Step 1: Check Dataset Information**

First inspect:

- Number of rows and columns
- Data types
- Missing values
- Descriptive statistics

`df.shape`

`df.info()`

`df.describe()`

`df.isnull().sum()`

This helps determine whether the dataset requires preprocessing.

### **Step 2: Check Correlation**

A correlation matrix can identify relationships between numerical variables.

Correlation values range from:

- $+1 \rightarrow$  strong positive relationship
- $0 \rightarrow$  little/no linear relationship
- $-1 \rightarrow$  strong negative relationship

For example, if two predictors have a correlation of 0.90 or above, they may create a multicollinearity problem.

### **Step 3: Calculate VIF**

**Variance Inflation Factor (VIF)** is another important method for detecting multicollinearity.

A commonly used interpretation is:

| VIF    | Interpretation             |
|--------|----------------------------|
| $< 5$  | Usually acceptable         |
| 5–10   | Possible multicollinearity |
| $> 10$ | Serious multicollinearity  |

Features with extremely high VIF can be removed or combined.

### **Step 4: Detect Outliers**

The **IQR method** can be used:

Lower boundary:

Upper boundary:

Observations outside these boundaries are potential outliers.

A box plot can visually identify these observations.

### **Step 5: Analyze Residuals**

A residual is:

If residuals are very large or show a systematic pattern, the model may not be capturing the relationship correctly.

A good regression model should have residuals approximately centered around zero without a strong pattern.

### **Step 6: Evaluate Model Performance**

The following metrics can be used:

Mean Absolute Error

Mean Squared Error

Root Mean Squared Error

$R^2$  Score

A lower MAE/RMSE and higher  $R^2$  generally indicate better performance.

### **(iii) Optimization Techniques:**

After identifying the problems, several optimization techniques can be applied.

#### **1. Feature Scaling**

Features can have very different ranges.

For example:

- YearBuilt → around 1900–2020
- GrLivArea → hundreds or thousands
- LotArea → thousands
- GarageCars → 0–4

Standardization transforms a feature approximately as:

This is particularly useful when using regularized linear models such as **Ridge** and **Lasso**.

#### **2. Remove or Reduce Outliers**

Potential extreme observations can be detected using the IQR method.

Instead of blindly removing every outlier, the observations should be investigated first.

If an observation is a genuine luxury house, it may be valid data rather than an error.

Therefore:

**Only remove observations when there is a reasonable justification.**

### 3. Feature Selection

Features with:

- very low correlation,
- excessive missing values,
- high VIF,
- or little predictive importance

can be removed.

Important house-price features can be retained, such as:

- OverallQual
- GrLivArea
- GarageCars
- GarageArea
- TotalBsmtSF
- 1stFlrSF
- YearBuilt
- YearRemodAdd
- FullBath
- TotRmsAbvGrd

**Dataset :**

<https://www.kaggle.com/c/house-prices-advanced-regression-techniques/data>

**Code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error, r2_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```

import warnings
warnings.filterwarnings("ignore")
print("Upload kc_house_data.csv")
uploaded = files.upload()
df = pd.read_csv("kc_house_data.csv")
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\n===== DATA INFORMATION =====")
print(df.info())
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())
print("\n===== DUPLICATE ROWS =====")
print("Duplicates:", df.duplicated().sum())
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
print("\n===== PRICE ANALYSIS =====")
print("Minimum Price:", df["price"].min())
print("Maximum Price:", df["price"].max())
print("Mean Price:", df["price"].mean())
print("Median Price:", df["price"].median())
print("Price Skewness:", df["price"].skew())
numeric_df = df.select_dtypes(include=np.number)
correlation = numeric_df.corr()
price_correlation = correlation["price"].sort_values(
    ascending=False
)
print("\n===== CORRELATION WITH PRICE =====")

print(price_correlation)
vif_features = [

```

```

    "bedrooms",
    "bathrooms",
    "sqft_living",
    "sqft_lot",
    "floors",
    "waterfront",
    "view",
    "condition",
    "grade",
    "sqft_above",
    "sqft_basement",
    "yr_built"
]
vif_df = df[vif_features].copy()
# Handle missing values
vif_df = vif_df.fillna(vif_df.median())
vif_result = pd.DataFrame()
vif_result["Feature"] = vif_df.columns
vif_result["VIF"] = [
    variance_inflation_factor(
        vif_df.values,
        i
    )
    for i in range(vif_df.shape[1])
]
vif_result = vif_result.sort_values(
    by="VIF",
    ascending=False
)

print("\n===== VIF ANALYSIS =====")
print(vif_result.to_string(index=False))

```

```

def count_outliers(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    count = (
        (data[column] < lower) |
        (data[column] > upper)
    ).sum()
    return count, lower, upper

price_outliers, price_lower, price_upper = \
    count_outliers(df, "price")
area_outliers, area_lower, area_upper = \
    count_outliers(df, "sqft_living")
print("\n===== OUTLIER ANALYSIS =====")
print("Price outliers:", price_outliers)
print("Price lower limit:", price_lower)
print("Price upper limit:", price_upper)
print("\nSqft_living outliers:", area_outliers)
print("Sqft_living lower limit:", area_lower)
print("Sqft_living upper limit:", area_upper)
original_size = len(df)
df = df[
    (df["price"] <= 2000000) &
    (df["sqft_living"] <= 5000)
].copy()
print("\n===== OUTLIER OPTIMIZATION =====")
print("Original records:", original_size)
print("Records after removing extreme outliers:", len(df))

print("Records removed:", original_size - len(df))
df["house_age"] = (
    df["yr_built"].max() - df["yr_built"]
)

```

```

df["total_area"] = (
    df["sqft_living"] +
    df["sqft_basement"]

df["rooms_per_bedroom"] = (
    df["sqft_living"] /
    (df["bedrooms"] + 1)

print("\n===== FEATURE ENGINEERING =====")
print("New features:")
print("1. house_age")
print("2. total_area")
print("3. rooms_per_bedroom")features = [
    "bedrooms",
    "bathrooms",
    "sqft_living",
    "sqft_lot",
    "floors",
    "waterfront",
    "view",
    "condition",
    "grade",
    "sqft_above",
    "sqft_basement",
    "yr_built",
    "yr_renovated",
    "house_age",
    "total_area",
    "rooms_per_bedroom"
]

X = df[features]
y = df["price"]X = X.fillna(X.median())

```

```

print("\nMissing values after preprocessing:")
print(X.isnull().sum().sum())
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("\n===== DATA SPLIT =====")
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
scaler = StandardScaler(
    _train_scaled = scaler.fit_transform(
        X_train
    )
X_test_scaled = scaler.transform(
    X_test
)
print("\nFeature scaling completed.")
linear_model = LinearRegression(
    linear_model.fit(
        X_train_scaled,
        y_train
    )
linear_prediction = linear_model.predict(
    X_test_scaled
)
ridge_model = Ridge(
    alpha=10
)

ridge_model.fit(
    X_train_scaled,

```

```

        y_train
    )
    ridge_prediction = ridge_model.predict(
        X_test_scaled
    )
    lasso_model = Lasso(
        alpha=100,
        max_iter=10000

    lasso_model.fit(
        X_train_scaled,
        y_train
    )
    lasso_prediction = lasso_model.predict(
        X_test_scaled
    )
    def evaluate_model(name, actual, predicted)
        mae = mean_absolute_error(
            actual,
            predicted
        )
        mse = mean_squared_error(
            actual,
            predicted
        )
        rmse = np.sqrt(mse)
        r2 = r2_score(
            actual,
            predicted
        )

    return [

```

```

        name,
        mae,
        rmse,
        r2
    ]
results = [
results.append(
    evaluate_model(
        "Linear Regression",
        y_test,
        linear_prediction
    )
)
results.append(
    evaluate_model(
        "Ridge Regression",
        y_test,
        ridge_prediction
    )
)
results.append(
    evaluate_model(
        "Lasso Regression",
        y_test,
        lasso_prediction
    )
)
results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "MAE",

```

```

        "RMSE",
        "R2 Score"
    ]
)print("\n===== MODEL PERFORMANCE =====")
print(
    results_df.to_string(
        index=False
    )
)
best_index = results_df["RMSE"].idxmin()

best_model = results_df.loc[
    best_index
]
print("\n===== BEST MODEL =====")
print("Best Model:", best_model["Model"])
print(
    "MAE:",
    round(best_model["MAE"], 2)
)
print(
    "RMSE:",
    round(best_model["RMSE"], 2)
)
print(
    "R2 Score:",
    round(best_model["R2 Score"], 4)
)

```

## Output:

```
Upload kc_house_data.csv
... Choose Files kc_house_data.csv
kc_house_data.csv(text/csv) - 2515206 bytes, last modified: 8/12/2026 - 100% done
Saving kc_house_data.csv to kc_house_data (1).csv

Dataset loaded successfully!
Dataset Shape: (21613, 21)

First 5 rows:
   id      date      price  bedrooms  bathrooms  sqft_living  \
0  7129300520  20141013T000000  221900.0         3         1.00        1180
1  6414100192  20141209T000000  538000.0         3         2.25        2570
2  5631500400  20150225T000000  180000.0         2         1.00         770
3  2487200875  20141209T000000  604000.0         4         3.00        1960
4  1954400510  20150218T000000  510000.0         3         2.00        1680

   sqft_lot  floors  waterfront  view  ...  grade  sqft_above  sqft_basement  \
0      5650     1.0           0     0  ...     7       1180           0
1      7242     2.0           0     0  ...     7       2170          400
2     10000     1.0           0     0  ...     6        770           0
3       5000     1.0           0     0  ...     7       1050          910
4       8080     1.0           0     0  ...     8       1680           0

   yr_built  yr_renovated  zipcode      lat      long  sqft_living15  \
0     1955           0     98178  47.5112 -122.257         1340
1     1951         1991     98125  47.7210 -122.319         1690
2     1933           0     98028  47.7379 -122.233         2720
3     1965           0     98136  47.5208 -122.393         1360
4     1987           0     98074  47.6168 -122.045         1800

   sqft_lot15
0           5650

... ===== DATA INFORMATION =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   id                    21613 non-null  int64
1   date                  21613 non-null  object
2   price                 21613 non-null  float64
3   bedrooms              21613 non-null  int64
4   bathrooms              21613 non-null  float64
5   sqft_living            21613 non-null  int64
6   sqft_lot               21613 non-null  int64
7   floors                21613 non-null  float64
8   waterfront             21613 non-null  int64
9   view                  21613 non-null  int64
10  condition              21613 non-null  int64
11  grade                  21613 non-null  int64
12  sqft_above             21613 non-null  int64
13  sqft_basement          21613 non-null  int64
14  yr_built               21613 non-null  int64
15  yr_renovated           21613 non-null  int64
16  zipcode                21613 non-null  int64
17  lat                    21613 non-null  float64
18  long                   21613 non-null  float64
19  sqft_living15          21613 non-null  int64
20  sqft_lot15             21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
None
```

```

===== DUPLICATE ROWS =====
... Duplicates: 0
Shape after removing duplicates: (21613, 21)

===== PRICE ANALYSIS =====
Minimum Price: 75000.0
Maximum Price: 7700000.0
Mean Price: 540088.1417665294
Median Price: 450000.0
Price Skewness: 4.024069144684712

===== CORRELATION WITH PRICE =====
price          1.000000
sqft_living    0.702035
grade          0.667434
sqft_above     0.605567
sqft_living15  0.585379
bathrooms      0.525138
view           0.397293
sqft_basement  0.323816
bedrooms       0.308350
lat            0.307003
waterfront     0.266369
floors         0.256794
yr_renovated   0.126434
sqft_lot       0.089661
sqft_lot15     0.082447
yr_built       0.054012
condition      0.036362
long           0.021626
id             -0.016762
zipcode        -0.053203

id             -0.016762
zipcode        -0.053203
... Name: price, dtype: float64

===== VIF ANALYSIS =====
Feature      VIF
sqft_living   inf
sqft_basement inf
sqft_above    inf
grade 128.744068
yr_built 126.311405
condition 30.016922
bathrooms 24.914810
bedrooms 22.996619
floors 16.017524
view 1.488292
waterfront 1.206842
sqft_lot 1.201946

===== OUTLIER ANALYSIS =====
Price outliers: 1146
Price lower limit: -162625.0
Price upper limit: 1129575.0

Sqft_living outliers: 572
Sqft_living lower limit: -257.5
Sqft_living upper limit: 4234.5

===== OUTLIER OPTIMIZATION =====
Original records: 21613
Records after removing extreme outliers: 21288
Records removed: 325

```

```

===== FEATURE ENGINEERING =====
New features:
1. house_age
2. total_area
3. rooms_per_bedroom

Missing values after preprocessing:
0

===== DATA SPLIT =====
Training records: 17030
Testing records: 4258

Feature scaling completed.

===== MODEL PERFORMANCE =====
      Model      MAE      RMSE  R2 Score
Linear Regression 123295.383873 172399.339587 0.621401
Ridge Regression 123275.367857 172303.780789 0.621821
Lasso Regression 123234.926751 172088.120055 0.622767

===== BEST MODEL =====
Best Model: Lasso Regression
MAE: 123234.93
RMSE: 172088.12
R2 Score: 0.6228

```

## Conclusion:

The high prediction error in the house-price Linear Regression model can be caused by multicollinearity, outliers, missing values, irrelevant features, feature-scale differences, and non-linear or skewed relationships. These problems can be detected using correlation analysis, VIF, box plots, residual analysis, and model evaluation metrics. The model can be improved through missing-value handling, outlier treatment, feature selection, feature engineering, standardization, logarithmic transformation, and regularization techniques such as Ridge and Lasso Regression. The optimized model is then selected based on lower RMSE and MAE and higher  $R^2$  score.

**Q2. An email classification system using a linear model is incorrectly classifying several spam emails as non-spam.**

- (i) Identify the reasons for misclassification (e.g., improper decision boundary, imbalanced data).**
- (ii) Examine the current decision boundary and classification errors.**
- (iii) Refine the model using optimization techniques such as adjusting thresholds, feature engineering, or re-training with balanced data.**

## Email Spam Classification Using a Linear Model

### Problem Statement

An email classification system uses a **linear classification model** to classify emails into **Spam** and **Non-Spam**. However, several spam emails are being classified as non-spam. This is a serious problem because unwanted or potentially harmful emails may reach the user's inbox.

When an actual spam email is predicted as non-spam, it is called a **False Negative (FN)**.

## **(i) Identify the Reasons for Misclassification**

### **1. Improper Decision Boundary**

A linear classifier separates spam and non-spam emails using a decision boundary.

If the boundary is not positioned correctly, some spam emails may fall on the non-spam side.

For example:

**Actual:**

Spam

**Predicted:**

Non-Spam

→ **False Negative**

Therefore, an unsuitable decision boundary can cause several spam emails to be missed.

### **2. Imbalanced Data**

Spam datasets are often imbalanced because the number of legitimate messages can be much larger than spam messages.

For example:

- Non-Spam → Majority class
- Spam → Minority class

A model trained directly on such data may become biased toward the majority class and classify uncertain messages as non-spam.

### **3. Poor Feature Representation**

The classifier needs useful features to identify spam.

Important spam-related features can include:

- free
- winner
- offer
- prize
- urgent
- claim
- click
- cash
- limited

If these features are not represented effectively, the classifier may fail to recognize spam.

#### 4. Incorrect Classification Threshold

A binary classifier commonly uses a threshold of **0.5**.

If the predicted spam probability is below 0.5, the message is classified as non-spam.

For example:

Spam probability = 0.45

Threshold = 0.50

Prediction = Non-Spam

Although the probability is fairly high, the message is classified as non-spam.

#### 5. Insufficient Training

If the training data does not contain enough examples of different types of spam, the model may not generalize well to new spam messages.

Spam messages can use different words, spelling variations, URLs, numbers, and promotional patterns.

##### (ii) Examine the Decision Boundary and Classification Errors

The current classifier can be analyzed using a **confusion matrix**.

|                 | Predicted Non-Spam | Predicted Spam |
|-----------------|--------------------|----------------|
| Actual Non-Spam | TN                 | FP             |
| Actual Spam     | FN                 | TP             |

The most important value for this problem is **FN (False Negative)**.

##### False Negative

A false negative occurs when:

**Actual Spam → Predicted Non-Spam**

A high FN value means the spam detection system is missing many spam messages.

#### Important Evaluation Metrics

**Accuracy**

**Precision**

**Recall**

**F1-score**

For this problem, **Recall for the spam class is particularly important**, because we want to reduce the number of spam emails incorrectly classified as non-spam

##### (iii) Refine the Model Using Optimization

###### 1. Adjust the Classification Threshold

Instead of using the default threshold:

we can reduce it to:

## 2. Feature Engineering

Convert email text into numerical features using **TF-IDF**.

TF-IDF gives higher importance to words that are useful for distinguishing messages.

We can also use:

- Unigrams
- Bigrams
- Word frequency
- Message length
- Number of special characters
- Number of URLs
- Number of capital letters

For this implementation, **TF-IDF with unigrams and bigrams** is used.

## 3. Balance the Training Data

The model can be trained using:

```
class_weight="balanced"
```

This automatically gives more importance to the minority class.

Therefore, spam messages receive greater importance during training.

### Dataset:

<https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

### Code:

```
import pandas as pd
import numpy as n
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
```

```

        classification_report
    )
import warnings
warnings.filterwarnings("ignore")
print("Upload your Kaggle spam.csv file")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(
    filename,
    encoding="latin-1"
)
print("\nDataset loaded successfully!")
print("Original Shape:", df.shape)
# v1 = label
# v2 = message

if "v1" in df.columns and "v2" in df.columns:
    df = df[["v1", "v2"]]
    df.columns = [
        "label",
        "message"
    ]
else:
    # If the uploaded dataset already has
    # label and message columns
    if "label" in df.columns and "message" in df.columns:
        df = df[["label", "message"]]
    else:
        # Use first two columns
        df = df.iloc[:, :2]
        df.columns = [
            "label",

```

```

        "message"
    ]
print("\nColumns used:")
print(df.columns.tolist())
print("\nMissing values before cleaning:")
print(df.isnull().sum())
df = df.dropna()
print("\nDataset after removing missing values:")
print(df.shape)
duplicate_count = df.duplicated().sum()
print("\nDuplicate messages:", duplicate_count)
df = df.drop_duplicates()
print(
    "Dataset after duplicate removal:",
    df.shape
)
print("\n===== CLASS DISTRIBUTION =====")

print(df["label"].value_counts())df["label"] = (
    df["label"]
    .astype(str)
    .str.lower()
    .str.strip()
)
df["label"] = df["label"].map({
    "ham": 0,
    "spam": 1,
    "0": 0,
    "1": 1
})
# Remove labels that could not be converted
df = df.dropna(subset=["label"])
df["label"] = df["label"].astype(int)

```

```

X = df["message"]
y = df["label"]X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
print("\n===== DATA SPLIT =====")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True
)
X_train_tfidf = vectorizer.fit_transform(
    X_train
)
X_test_tfidf = vectorizer.transform(
    X_test
)
print("\n===== FEATURE ENGINEERING =====")
print(
    "Number of TF-IDF features:",
    X_train_tfidf.shape[1]
)
original_model = LogisticRegression(
    max_iter=2000
)

```

```

original_model.fit(
    X_train_tfidf,
    y_train
)
spam_probability = (
    original_model
    .predict_proba(X_test_tfidf)[:, 1]
)
default_prediction = (
    spam_probability >= 0.50
).astype(int)
def evaluate_model(
    name,
    actual,
    predicted
):

    cm = confusion_matrix(
        actual,
        predicted
    )
    tn, fp, fn, tp = cm.ravel()
    accuracy = accuracy_score(
        actual,
        predicted
    )
    precision = precision_score(
        actual,
        predicted,
        zero_division=0
    )
    recall = recall_score(

```

```

        actual,
        predicted,
        zero_division=0
    )
    f1 = f1_score(
        actual,
        predicted,
        zero_division=0
    )
    return [
        name,
        accuracy,
        precision,
        recall,
        f1,
        tn,
        fp,
        fn,
        tp
    ]
default_result = evaluate_model(
    "Original Threshold 0.50",
    y_test,
    default_prediction
)
print("\n===== ORIGINAL MODEL =====")
print("Accuracy:", round(default_result[1], 4))
print("Precision:", round(default_result[2], 4))
print("Recall:", round(default_result[3], 4))
print("F1 Score:", round(default_result[4], 4))
print("\nTN:", default_result[5])
print("FP:", default_result[6])

```

```

print("FN:", default_result[7])
print("TP:", default_result[8])
thresholds = [
    0.50,
    0.45,
    0.40,
    0.35,
    0.30,
    0.25
]
threshold_results = []
for threshold in thresholds:
    prediction = (
        spam_probability >= threshold
    ).astype(int)

    result = evaluate_model(
        "Threshold " + str(threshold),
        y_test,
        prediction
    )
    threshold_results.append(result)
threshold_df = pd.DataFrame(
    threshold_results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1",
        "TN",
        "FP",
    ]
)

```

```

        "FN",
        "TP"
    ]
)
print("\n===== THRESHOLD OPTIMIZATION =====")
print(
    threshold_df.to_string(
        index=False
    )
)
balanced_model = LogisticRegression(
    max_iter=2000,
    class_weight="balanced"
)
balanced_model.fit(
    X_train_tfidf,
    y_train
)
balanced_probability = (
    balanced_model
    .predict_proba(X_test_tfidf)[:, 1]
)
balanced_results = [
    fo threshold in thresholds
        prediction = (
            balanced_probability >= threshold
        ).astype(int)
        result = evaluate_model(
            "Balanced " + str(threshold),
            y_test,
            prediction
        )
        balanced_results.append(result)
balanced_df = pd.DataFrame(

```

```

balanced_results,
columns=[
    "Model",
    "Accuracy",
    "Precision",
    "Recall",
    "F1",
    "TN",
    "FP",
    "FN",
    "TP"
]
print("\n===== BALANCED MODEL RESULTS =====")
print(
    balanced_df.to_string(
        index=False
    )
)
best_index = balanced_df["F1"].idxmax(
best_result = balanced_df.loc[
    best_index
]
print("\n===== BEST OPTIMIZED MODEL =====")
print(
    "Model:",
    best_result["Model"]
)
print(
    "Accuracy:",
    round(best_result["Accuracy"], 4)
)
print(

```

```
"Precision:",
round(best_result["Precision"], 4)

print(
    "Recall:",
    round(best_result["Recall"], 4)
)
print(
    "F1 Score:",
    round(best_result["F1"], 4)

print(
    "True Negatives:",
    int(best_result["TN"])
)

print(
    "False Positives:",
    int(best_result["FP"])
)
print(
    "False Negatives:",
    int(best_result["FN"])
)
print(
    "True Positives:",
    int(best_result["TP"])
)
best_threshold_text = best_result["Model"]
best_threshold = float(
    best_threshold_text.split()[-1]
)
```

```

final_prediction = (
    balanced_probability >= best_threshold
).astype(int)
print("\n===== FINAL CLASSIFICATION REPORT =====")
print(
    classification_report(
        y_test,
        final_prediction,
        target_names=[
            "Non-Spam",
            "Spam"
        ],
        zero_division=0
    )
)
final_cm = confusion_matrix(
    y_test,
    final_prediction

```

```

tn, fp, fn, tp = final_cm.ravel()
print("\n===== FINAL CONFUSION MATRIX =====")
print(final_cm)
print("\nTrue Negatives:", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives:", tp)

```

## Output:

```

... Upload your Kaggle spam.csv file
Choose Files spam.csv
spam.csv(text/csv) - 503663 bytes, last modified: 8/12/2026 - 100% done
Saving spam.csv to spam.csv

Dataset loaded successfully!
Original Shape: (5572, 5)

Columns used:
['label', 'message']

Missing values before cleaning:
label      0
message    0
dtype: int64

Dataset after removing missing values:
(5572, 2)

Duplicate messages: 403
Dataset after duplicate removal: (5169, 2)

===== CLASS DISTRIBUTION =====
label
ham    4516
spam    653
Name: count, dtype: int64

===== DATA SPLIT =====
Training samples: 4135
Testing samples: 1034

```

```

***
===== FEATURE ENGINEERING =====
Number of TF-IDF features: 5632

===== ORIGINAL MODEL =====
Accuracy: 0.9574
Precision: 0.9888
Recall: 0.6718
F1 Score: 0.8

TN: 902
FP: 1
FN: 43
TP: 88

===== THRESHOLD OPTIMIZATION =====
Model Accuracy Precision Recall F1 TN FP FN TP
Threshold 0.5 0.957447 0.988764 0.671756 0.800000 902 1 43 88
Threshold 0.45 0.966151 0.989796 0.740458 0.847162 902 1 34 97
Threshold 0.4 0.971954 0.981132 0.793893 0.877637 901 2 27 104
Threshold 0.35 0.971954 0.972222 0.801527 0.878661 900 3 26 105
Threshold 0.3 0.975822 0.956897 0.847328 0.898785 898 5 20 111
Threshold 0.25 0.979691 0.950820 0.885496 0.916996 897 6 15 116

===== BALANCED MODEL RESULTS =====
Model Accuracy Precision Recall F1 TN FP FN TP
Balanced 0.5 0.976789 0.928000 0.885496 0.906250 894 9 15 116
Balanced 0.45 0.974855 0.894737 0.908397 0.901515 889 14 12 119
Balanced 0.4 0.972921 0.860140 0.938931 0.897810 883 20 8 123
Balanced 0.35 0.966151 0.811688 0.954198 0.877193 874 29 6 125
Balanced 0.3 0.958414 0.761905 0.977099 0.856187 863 40 3 128

***
===== BEST OPTIMIZED MODEL =====
Model: Balanced 0.5
Accuracy: 0.9768
Precision: 0.928
Recall: 0.8855
F1 Score: 0.9062
True Negatives: 894
False Positives: 9
False Negatives: 15
True Positives: 116

===== FINAL CLASSIFICATION REPORT =====
precision recall f1-score support

Non-Spam 0.98 0.99 0.99 903
Spam 0.93 0.89 0.91 131

accuracy 0.98 1034
macro avg 0.96 0.94 0.95 1034
weighted avg 0.98 0.98 0.98 1034

===== FINAL CONFUSION MATRIX =====
[[894 9]
 [ 15 116]]

True Negatives: 894
False Positives: 9
False Negatives: 15
True Positives: 116

```

## Conclusion:

The misclassification of spam emails as non-spam can occur due to an improper decision boundary, class imbalance, weak feature representation, unsuitable classification threshold, and insufficient training examples. The problem can be analyzed using a confusion matrix, particularly by examining False Negatives, along with precision, recall, F1-score and accuracy. The model can be improved by using TF-IDF and n-gram feature engineering, balancing the training classes, and reducing the classification threshold from 0.5 to a lower value such as 0.4. The optimized model is then evaluated using the confusion matrix and classification metrics.

The main objective is to reduce False Negatives and increase spam recall while maintaining acceptable precision.

**Q3. A Naïve Bayes classifier is giving poor accuracy due to missing or zero probability values in the dataset.**

- (i) Identify the issue related to missing or zero probabilities.**
- (ii) Analyze how these values affect classification results.**
- (iii) Apply appropriate debugging and optimization methods such as Laplace smoothing to improve accuracy.**

### **Naïve Bayes Classifier – Zero Probability Problem and Laplace Smoothing**

#### **Problem Statement:**

A **Naïve Bayes classifier** is giving poor classification accuracy because some feature values are missing or have **zero probability** in the training dataset. This is known as the **zero-frequency problem**.

For example, suppose the classifier is trained to identify **spam and non-spam messages**. If a particular word does not occur in the training examples of the spam class, its conditional probability may become zero.

#### **(i) Identify the Issue Related to Missing or Zero Probabilities**

Naïve Bayes calculates the probability of a class using:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$

$$P(A)$$

$$P(A)$$

$$P(B | A)$$

$$P(B | A)$$

$$P(B | \neg A)$$

$$P(B | \neg A)$$

$$P(B)=0.25P(B|A)P(A)=0.17P(A|B)\sim0.68$$

Posterior = useful evidence / total evidence

For multiple features:  $P(C|X) \propto P(C) \times P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$

Suppose:

$$P(\text{"offer"} | \text{Spam}) = 0$$

Then:

Therefore:

Even if all the other features strongly indicate spam, the complete probability becomes zero.

### **Main Issues**

1. Missing feature values
2. Features that never occur in a particular class
3. Small training datasets
4. Rare words or categories
5. High-dimensional text features
6. Zero conditional probabilities

This is called the **zero-frequency problem**.

### **(ii) How Zero Probabilities Affect Classification**

Naïve Bayes multiplies conditional probabilities.

Therefore, even **one zero probability can make the entire class probability zero**.

### **Effect on Classification**

Zero probabilities can cause:

- Incorrect class probabilities
- Wrong predictions
- Poor accuracy
- Increased false classifications
- Poor recall
- Unstable predictions for rare features

### **(iii) Debugging and Optimization Using Laplace Smoothing**

The main solution is **Laplace smoothing**, also called **add-one smoothing**.

Instead of using:

we use:

where:

- $\text{Count}(\text{word}, \text{class})$  = number of times the word occurs in that class
- $\text{TotalCount}(\text{class})$  = total number of words in the class
- $V$  = number of unique words in the vocabulary

Adding 1 ensures that no probability becomes zero.

### **Other Optimization Techniques**

#### **1. Handle Missing Values**

Missing values should be identified before training.

Possible approaches:

- Remove unnecessary missing records
- Replace missing numerical values with median
- Replace missing categorical values with mode

## **2. Feature Engineering**

For text classification, use:

- Bag of Words
- TF-IDF
- Unigrams
- Bigrams

## **3. Remove Extremely Rare Features**

Words that occur only once or twice may introduce noise.

`min_df` can be used to remove very rare features.

## **4. Tune the Smoothing Parameter**

In MultinomialNB, the smoothing parameter is controlled by `alpha`.

Examples:

`alpha = 0.1`

`alpha = 0.5`

`alpha = 1.0`

`alpha = 2.0`

The best value can be selected based on validation accuracy or F1-score.

### **Dataset:**

Kaggle – SMS Spam Collection Dataset

<https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

### **code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import warnings
warnings.filterwarnings("ignore")
print("Upload spam.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(
    filename,
    encoding="latin-1"
)
print("\nDataset loaded successfully!")
print("Original Shape:", df.shape)if "v1" in df.columns and "v2" in df.columns:
    df = df[["v1", "v2"]]
    df.columns = [
        "label",
        "message"

else:

    if "label" in df.columns and "message" in df.columns:

        df = df[["label", "message"]]
    else:
        df = df.iloc[:, :2]
        df.columns = [

```

```

        "label",
        "message"
    ]
print("\nColumns used:")
print(df.columns.tolist())
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())
df = df.dropna(
    subset=["message"]
print("\nDuplicate records:",
    df.duplicated().sum())
df = df.drop_duplicates()
print(
    "Shape after duplicate removal:",
    df.shape
)
print("\n===== CLASS DISTRIBUTION =====")
print(
    df["label"].value_counts()
)
df["label"] = (
    df["label"]
    .astype(str)
    .str.lower()
    .str.strip()
)

df["label"] = df["label"].map({
    "ham": 0,
    "spam": 1,
    "0": 0,
    "1": 1

```

```

})
df = df.dropna(
    subset=["label"]
)
df["label"] = df[
    "label"
].astype(int)
X = df["message"]
y = df["label"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("\n===== DATA SPLIT =====")

print(
    "Training samples:",
    len(X_train)
)
print(
    "Testing samples:",
    len(X_test)
)

vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2

```

```

)
X_train_count = vectorizer.fit_transform(
    X_train
)
X_test_count = vectorizer.transform(
    X_test

print("\n===== FEATURE EXTRACTION =====")
print(
    "Vocabulary size:",
    len(vectorizer.vocabulary_)
)
print("\n===== WITHOUT SMOOTHING =====")
try:
    nb_without = MultinomialNB(
        alpha=0
    )
    nb_without.fit(
        X_train_count,
        y_train
    )
    prediction_without = nb_without.predict(
        X_test_count
    )
    accuracy_without = accuracy_score(
        y_test,
        prediction_without
    )
    print(
        "Accuracy:",
        round(accuracy_without, 4)
    )

```

```
except Exception as e:
    print(
        "Zero-probability model encountered:"
    )
    print(e)
nb_smoothing = MultinomialNB(
    alpha=1.0
)
nb_smoothing.fit(
    X_train_count,
    y_train
)
prediction_smoothing = nb_smoothing.predict(
    X_test_count
)
accuracy = accuracy_score(
    y_test,
    prediction_smoothing
)
precision = precision_score(
    y_test,
    prediction_smoothing,
    zero_division=0
)
recall = recall_score(
    y_test,
    prediction_smoothing,
    zero_division=0
)
f1 = f1_score(
    y_test,
    prediction_smoothing,
```

```

    zero_division=0
)
print("\n===== LAPLACE SMOOTHING =====")
print(
    "Alpha:",
    1.0
)
print(
    "Accuracy:",
    round(accuracy, 4)
)
print(
    "Precision:",
    round(precision, 4)
)
print(
    "Recall:",
    round(recall, 4)
)
print(
    "F1 Score:",
    round(f1, 4)
)
cm = confusion_matrix(
    y_test,
    prediction_smoothing
)

print("\n===== CONFUSION MATRIX =====")
print(cm)
alphas = [
    0.01,

```

```

0.1,
0.5,
1.0,
1.5,
2.0,
5.0
]
results = [
for alpha in alphas
    model = MultinomialNB(
        alpha=alpha
    )
    model.fit(
        X_train_count,
        y_train
    )
    prediction = model.predict(
        X_test_count
    )
    accuracy_value = accuracy_score(
        y_test,
        prediction
    )
    precision_value = precision_score(
        y_test,
        prediction,
        zero_division=0
    )
    recall_value = recall_score(
        y_test,
        prediction,
        zero_division=0

```

```

    )
    fl_value = f1_score(
        y_test,
        prediction,
        zero_division=0
    )
    results.append([
        alpha,
        accuracy_value,
        precision_value,
        recall_value,
        fl_value
    ])
results_df = pd.DataFrame(
    results,
    columns=[
        "Alpha",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(
    "\n===== ALPHA COMPARISON ====="
)

print(
    results_df.to_string(
        index=False
    )
)

```

```

best_index = results_df[
    "F1 Score"
].idxmax()
best_alpha = results_df.loc[
    best_index,
    "Alpha"
]
best_accuracy = results_df.loc[
    best_index,
    "Accuracy"

best_precision = results_df.loc[
    best_index,
    "Precision"
]
est_recall = results_df.loc[
    best_index,
    "Recall"
]
best_f1 = results_df.loc[
    best_index,
    "F1 Score"
]
print(
    "\n===== BEST SMOOTHING PARAMETER ====="
)

print(
    "Best Alpha:",
    best_alpha
)
print(

```

```

    "Accuracy:",
    round(best_accuracy, 4)
)
print(
    "Precision:",
    round(best_precision, 4)
)
print(
    "Recall:",
    round(best_recall, 4)
)
print(
    "F1 Score:",
    round(best_f1, 4)
)
final_model = MultinomialNB(
    alpha=best_alpha
)
final_model.fit(
    X_train_count,
    y_train
)
final_prediction = final_model.predict(
    X_test_count
)
final_cm = confusion_matrix(
    y_test,
    final_prediction
)
print(
    "\n===== FINAL CONFUSION MATRIX ====="
)

```

```

print(final_cm)
print(
    "\n===== FINAL CLASSIFICATION REPORT ====="
)
print(
    classification_report(
        y_test,
        final_prediction,
        target_names=[
            "Non-Spam",
            "Spam"
        ],
        zero_division=0
    )
)

```

## Output:

```

Upload spam.csv
... Choose Files spam.csv
spam.csv(text/csv) - 503663 bytes, last modified: 8/12/2026 - 100% done
Saving spam.csv to spam (1).csv

Dataset loaded successfully!
Original Shape: (5572, 5)

Columns used:
['label', 'message']

===== MISSING VALUES =====
label      0
message    0
dtype: int64

Duplicate records: 403
Shape after duplicate removal: (5169, 2)

===== CLASS DISTRIBUTION =====
label
ham      4516
spam      653
Name: count, dtype: int64

===== DATA SPLIT =====
Training samples: 4135
Testing samples: 1034

===== FEATURE EXTRACTION =====
Vocabulary size: 5632

```

```

===== WITHOUT SMOOTHING =====
*** Accuracy: 0.9555

===== LAPLACE SMOOTHING =====
Alpha: 1.0
Accuracy: 0.9845
Precision: 0.9752
Recall: 0.9008
F1 Score: 0.9365

===== CONFUSION MATRIX =====
[[900   3]
 [ 13 118]]

===== ALPHA COMPARISON =====
Alpha  Accuracy  Precision  Recall  F1 Score
0.01   0.983559   0.959677  0.908397  0.933333
0.10   0.984526   0.960000  0.916031  0.937500
0.50   0.983559   0.959677  0.908397  0.933333
1.00   0.984526   0.975207  0.900763  0.936508
1.50   0.984526   0.975207  0.900763  0.936508
2.00   0.985493   0.983333  0.900763  0.940239
5.00   0.979691   0.982456  0.854962  0.914286

===== BEST SMOOTHING PARAMETER =====
Best Alpha: 2.0
Accuracy: 0.9855
Precision: 0.9833
Recall: 0.9008
F1 Score: 0.9402

===== FINAL CONFUSION MATRIX =====
[[901   2]
 [ 13 118]]

===== FINAL CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

   Non-Spam       0.99         1.00         0.99         903
     Spam        0.98         0.90         0.94         131

   accuracy                   0.99         1034
  macro avg       0.98         0.95         0.97         1034
 weighted avg     0.99         0.99         0.99         1034

```

## Conclusion

The poor performance of the Naïve Bayes classifier was caused by missing or zero probability values, also known as the zero-frequency problem. A zero probability for even one feature can make the overall class probability zero and lead to incorrect classification.

The problem was addressed using data cleaning, feature extraction, and Laplace smoothing. By adding a small smoothing value to feature counts, zero probabilities are avoided. Different alpha values were tested to identify a suitable smoothing parameter.

Thus, Laplace smoothing improves the reliability and stability of the Naïve Bayes classifier and can improve its classification performance, particularly when the dataset contains rare or previously unseen features.

**Q4. A discriminant function is unable to clearly separate two classes, leading to incorrect classifications.**

- (i) Identify possible reasons for poor class separation (e.g., overlapping features, incorrect weights).**
- (ii) Analyze the decision boundary and feature contribution.**
- (iii) Improve the discriminant function using optimization strategies such as adjusting weights, feature transformation, or normalization.**

## **Discriminant Function – Poor Class Separation**

### **Problem Statement**

A **discriminant function** is used to separate two classes and assign a new data point to the appropriate class. However, the current function is unable to clearly separate the two classes, resulting in incorrect classifications.

A linear discriminant function can be represented as:

where:

- = input features
- = feature weights
- = bias/intercept
- = discriminant score

The decision is generally made as:

$D(X) > 0 \rightarrow \text{Class 1}$

$D(X) < 0 \rightarrow \text{Class 2}$

### **(i) Reasons for Poor Class Separation**

#### **1. Overlapping Features**

The two classes may have similar feature values. When their feature distributions overlap significantly, a single linear boundary cannot separate them accurately.

#### **2. Incorrect Weights**

The discriminant function depends on feature weights. If the weights are poorly estimated, important features may receive low weights while less important features receive high weights.

#### **3. Different Feature Scales**

Features may have very different numerical ranges.

For example:

Feature 1  $\rightarrow$  0–10

Feature 2  $\rightarrow$  0–1000

The larger-scale feature can dominate the discriminant function.

#### **4. Irrelevant Features**

Unimportant features can introduce noise and make the decision boundary less effective.

#### **5. Insufficient Training Data**

If the training data does not represent both classes properly, the model may learn an unsuitable boundary.

#### **6. Non-Linear Class Separation**

Sometimes the classes cannot be separated effectively using a straight-line/linear boundary. In such cases, feature transformation may be required.

#### **(ii) Analyze the Decision Boundary and Feature Contribution**

The decision boundary occurs when:

$$|-3| = 3$$

$$|x| = -x$$

Distance from 0

–6–5–4–3–2–10123456

Drag the point left or right

Give feedback

The following should be examined:

#### **1. Confusion Matrix**

The confusion matrix identifies:

- True Positives
- True Negatives
- False Positives
- False Negatives

A large number of incorrect predictions indicates poor class separation.

#### **2. Accuracy**

Measures the percentage of correctly classified observations.

#### **3. Precision and Recall**

These identify how effectively each class is detected.

#### **4. Feature Weights**

The learned coefficients can be examined to understand feature contribution.

For example:

| Feature   | Weight | Interpretation               |
|-----------|--------|------------------------------|
| Feature 1 | 2.45   | Strong positive contribution |
| Feature 2 | -1.82  | Strong negative contribution |
| Feature 3 | 0.15   | Small contribution           |
| Feature 4 | 0.02   | Very small contribution      |

A larger absolute coefficient generally indicates a stronger contribution **when features have been normalized**.

### (iii) Improve the Discriminant Function

#### 1. Feature Normalization

Standardization puts features on a comparable scale:

This prevents large-scale features from dominating the discriminant function.

#### 2. Adjust Feature Weights

The model can be retrained to learn better weights.

Regularization can also prevent excessively large weights.

#### 3. Feature Selection

Remove irrelevant or highly noisy features and retain the variables that provide stronger class discrimination.

#### 4. Feature Transformation

If the classes are not linearly separable, transform the features using techniques such as:

- Polynomial features
- Log transformation
- Interaction features

This can make the classes easier to separate.

#### 5. Class Balancing

If one class contains substantially more observations than the other, balanced class weights can be used during training.

#### 6. Regularization

Regularization controls the size of model coefficients and reduces overfitting.

#### Dataset:

Kaggle – Breast Cancer Wisconsin Diagnostic Dataset

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

code:

```

import pandas as pd
import numpy as np
from google.colab import file
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import warnings
warnings.filterwarnings("ignore")
print("Upload data.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
if "Unnamed: 32" in df.columns:
    df = df.drop(
        columns=["Unnamed: 32"]
    )
if "id" in df.columns:
    df = df.drop(
        columns=["id"]
    )

```

```

df["diagnosis"] = df[
    "diagnosis"
].map({
    "M": 1,
    "B": 0
})
print("\n===== CLASS DISTRIBUTION =====")
print(
    df["diagnosis"].value_counts()
)
X = df.drop(
    columns=["diagnosis"]
)
y = df["diagnosis"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("\n===== DATA SPLIT =====")
print(
    "Training samples:",
    len(X_train)
)
print(
    "Testing samples:",
    len(X_test)
)

```

```
original_model = LogisticRegression(
    max_iter=5000
)
original_model.fit(
    X_train,
    y_train
)
original_prediction = (
    original_model.predict(
        X_test
    )
)
original_accuracy = accuracy_score(
    y_test,
    original_prediction

original_precision = precision_score(
    y_test,
    original_prediction,
    zero_division=0
)
original_recall = recall_score(
    y_test,
    original_prediction,
    zero_division=0
)
original_f1 = f1_score(
    y_test,
    original_prediction,
    zero_division=0
)
print("\n===== ORIGINAL MODEL =====")
```

```

print(
    "Accuracy:",
    round(original_accuracy, 4)
)
print(
    "Precision:",
    round(original_precision, 4)
)
print(
    "Recall:",
    round(original_recall, 4)
)
print(
    "F1 Score:",
    round(original_f1, 4)
)
print("\n===== ORIGINAL CONFUSION MATRIX =====")
print(
    confusion_matrix(
        y_test,
        original_prediction
    )
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(
    X_train
)
X_test_scaled = scaler.transform(
    X_test
)
print("\nFeature normalization completed.")

```

```
optimized_model = LogisticRegression(  
    max_iter=5000,  
    C=1.0,  
    class_weight="balanced"  
)  
optimized_model.fit(  
    X_train_scaled,  
    y_train  
)  
optimized_prediction = (  
    optimized_model.predict(  
        X_test_scaled  
    )  
)  
optimized_accuracy = accuracy_score(  
    y_test,  
    optimized_prediction  
  
optimized_precision = precision_score(  
    y_test,  
    optimized_prediction,  
    zero_division=0  
)  
optimized_recall = recall_score(  
    y_test,  
    optimized_prediction,  
    zero_division=0  
)  
optimized_f1 = f1_score(  
    y_test,  
    optimized_prediction,  
    zero_division=0
```

```

)
print("\n===== OPTIMIZED MODEL =====")
print(
    "Accuracy:",
    round(optimized_accuracy, 4)
)
print(
    "Precision:",
    round(optimized_precision, 4)
)
print(
    "Recall:",
    round(optimized_recall, 4)
)
print(
    "F1 Score:",
    round(optimized_f1, 4)
)
confusion_matrix(
    y_test,
    optimized_prediction
)
)
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Weight": optimized_model.coef_[0]
})
coefficients["Absolute_Weight"] = (
    coefficients["Weight"].abs()
)
coefficients = coefficients.sort_values(
    by="Absolute_Weight",

```

```

        ascending=False
    )
    print("\n===== FEATURE CONTRIBUTION =====")

    print(
        coefficients.to_string(
            index=False
        )
    )
)
C_values = [
    0.001,
    0.01,
    0.1,
    1.0,
    10.0,
    100.0
]
results = []
for c in C_values:
    model = LogisticRegression(
        C=c,
        max_iter=5000,
        class_weight="balanced"
    )
    model.fit(
        X_train_scaled,
        y_train
    )

    prediction = model.predict(
        X_test_scaled

```

```
accuracy = accuracy_score(  
    y_test,  
    prediction  
)
```

```
precision = precision_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
f1 = f1_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
results.append([  
    c,  
    accuracy,  
    precision,  
    recall,  
    f1  
)
```

```
results_df = pd.DataFrame(  
    results,  
    columns=[
```

```

        "C",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)
print(
    "\n===== REGULARIZATION COMPARISON ====="
)

print(
    results_df.to_string(
        index=False
    )
)
best_index = results_df[
    "F1 Score"
].idxmax()

best_C = results_df.loc[
    best_index,
    "C"
]
best_accuracy = results_df.loc[
    best_index,
    "Accuracy"
]

best_precision = results_df.loc[
    best_index,
    "Precision"
]

```

```

]
bestrecall = results_df.loc[
    best_index,
    "Recall"
]
print(
    "\n===== FINAL CLASSIFICATION REPORT ====="
)
print(
    classification_report(
        y_test,
        final_prediction,
        target_names=[
            "Benign",
            "Malignant"
        ],
        zero_division=0
    )
)

```

## Output:

... Upload data.csv

data.csv

**data.csv**(text/csv) - 125204 bytes, last modified: 8/12/2026 - 100% done

Saving data.csv to data.csv

Dataset loaded successfully!

Dataset Shape: (569, 33)

First 5 rows:

|   | id       | diagnosis | radius_mean | texture_mean | perimeter_mean | area_mean | \ |
|---|----------|-----------|-------------|--------------|----------------|-----------|---|
| 0 | 842302   | M         | 17.99       | 10.38        | 122.80         | 1001.0    |   |
| 1 | 842517   | M         | 20.57       | 17.77        | 132.90         | 1326.0    |   |
| 2 | 84300903 | M         | 19.69       | 21.25        | 130.00         | 1203.0    |   |
| 3 | 84348301 | M         | 11.42       | 20.38        | 77.58          | 386.1     |   |
| 4 | 84358402 | M         | 20.29       | 14.34        | 135.10         | 1297.0    |   |

|   | smoothness_mean | compactness_mean | concavity_mean | concave points_mean | \ |
|---|-----------------|------------------|----------------|---------------------|---|
| 0 | 0.11840         | 0.27760          | 0.3001         | 0.14710             |   |
| 1 | 0.08474         | 0.07864          | 0.0869         | 0.07017             |   |
| 2 | 0.10960         | 0.15990          | 0.1974         | 0.12790             |   |
| 3 | 0.14250         | 0.28390          | 0.2414         | 0.10520             |   |
| 4 | 0.10030         | 0.13280          | 0.1980         | 0.10430             |   |

|   | ... | texture_worst | perimeter_worst | area_worst | smoothness_worst | \ |
|---|-----|---------------|-----------------|------------|------------------|---|
| 0 | ... | 17.33         | 184.60          | 2019.0     | 0.1622           |   |
| 1 | ... | 23.41         | 158.80          | 1956.0     | 0.1238           |   |
| 2 | ... | 25.53         | 152.50          | 1709.0     | 0.1444           |   |
| 3 | ... | 26.50         | 98.87           | 567.7      | 0.2098           |   |
| 4 | ... | 16.67         | 152.20          | 1575.0     | 0.1374           |   |

```

compactness_worst concavity_worst concave points_worst symmetry_worst \
0      0.6656      0.7119      0.2654      0.4601
1      0.1866      0.2416      0.1860      0.2750
2      0.4245      0.4504      0.2430      0.3613
3      0.8663      0.6869      0.2575      0.6638
4      0.2050      0.4000      0.1625      0.2364

fractal_dimension_worst Unnamed: 32
0      0.11890      NaN
1      0.08902      NaN
2      0.08758      NaN
3      0.17300      NaN
4      0.07678      NaN

[5 rows x 33 columns]

===== CLASS DISTRIBUTION =====
diagnosis
0      357
1      212
Name: count, dtype: int64

===== MISSING VALUES =====
0

===== DATA SPLIT =====
Training samples: 455
Testing samples: 114

===== BEST MODEL =====
Best C: 0.1
Accuracy: 0.9912
Precision: 1.0
Recall: 0.9762
F1 Score: 0.988

===== FINAL CLASSIFICATION REPORT =====

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Benign       | 0.99      | 1.00   | 0.99     | 72      |
| Malignant    | 1.00      | 0.98   | 0.99     | 42      |
| accuracy     |           |        | 0.99     | 114     |
| macro avg    | 0.99      | 0.99   | 0.99     | 114     |
| weighted avg | 0.99      | 0.99   | 0.99     | 114     |

## Conclusion:

The poor class separation was caused by overlapping feature values, inappropriate feature weights, different feature scales, and irrelevant or less useful features. The decision boundary and feature contributions were analyzed using classification performance, confusion matrix, and model coefficients.

The discriminant function was improved by applying feature normalization, balanced class weights, feature contribution analysis, and regularization. Different regularization values were tested to identify a better set of weights. These optimization techniques help create a more effective decision boundary, reduce incorrect classifications, and improve the overall classification performance of the model.

**Q5.A Bayesian logistic regression model developed for customer churn prediction is suffering from overfitting, resulting in poor generalization.**

**(i) Identify signs and causes of overfitting in the model.**

(ii) Analyze model performance on training vs testing data.

(iii) Apply debugging and regularization techniques (such as L1/L2 regularization or cross-validation) to improve model performance

## **Bayesian Logistic Regression for Customer Churn Prediction**

### **Problem Statement**

A **Bayesian Logistic Regression model** is developed to predict whether a customer will **churn or remain with the company**. However, the model is suffering from **overfitting**, meaning it performs very well on training data but performs poorly on unseen testing data.

The main objective is to identify the overfitting problem, compare training and testing performance, and improve generalization using **L1/L2 regularization and cross-validation**.

### **(i) Signs and Causes of Overfitting**

#### **Signs of Overfitting**

##### **1. High Training Accuracy but Low Testing Accuracy**

For example:

Training Accuracy = 95%

Testing Accuracy = 78%

The large difference indicates that the model has learned the training data too closely.

##### **2. Training Error is Very Low**

The model makes very few errors on training data but produces significantly more errors on new data.

##### **3. Large Training–Testing Performance Gap**

We can calculate:

A large gap indicates poor generalization.

##### **4. Poor Testing F1/AUC**

The model may have high training F1-score or ROC-AUC but substantially lower testing values.

#### **Causes of Overfitting**

##### **1. Too Many Features**

Including unnecessary customer attributes can make the model learn noise.

##### **2. Complex Model**

A model with excessive parameters can fit individual training observations rather than learning general patterns.

##### **3. Insufficient Regularization**

Without adequate regularization, regression coefficients can become unnecessarily large.

#### 4. Noise in the Dataset

Incorrect, inconsistent, or noisy observations can cause the model to learn patterns that do not generalize.

#### 5. Small or Unrepresentative Training Data

If the training data does not represent the overall customer population, the model may perform poorly on unseen customers.

#### 6. Data Leakage

Information that would not actually be available when predicting churn may accidentally enter the training features.

#### (ii) Analyze Training vs Testing Performance

The model should be evaluated separately on **training and testing datasets**.

| Metric    | Training  | Testing   |
|-----------|-----------|-----------|
| Accuracy  | Calculate | Calculate |
| Precision | Calculate | Calculate |
| Recall    | Calculate | Calculate |
| F1-score  | Calculate | Calculate |
| ROC-AUC   | Calculate | Calculate |

#### (iii) Debugging and Regularization Techniques

##### 1. L1 Regularization

L1 regularization adds a penalty based on the absolute value of coefficients:

It can force some coefficients to become zero.

Therefore, L1 regularization can perform **automatic feature selection**.

##### 2. L2 Regularization

L2 regularization adds a squared coefficient penalty:

It reduces excessively large coefficients and produces a more stable model.

L2 is useful when several customer features contribute to churn prediction.

##### 3. Cross-Validation

Instead of relying on a single train-test split, **K-fold cross-validation** divides the training data into multiple folds.

For example, with 5-fold cross-validation:

Fold 1 → Testing

Fold 2–5 → Training

Fold 2 → Testing

Fold 1,3–5 → Training

...

The average performance across folds provides a more reliable estimate of model performance.

#### 4. Feature Selection

Remove unnecessary features and retain variables that have meaningful relationships with churn.

Examples may include:

- Contract type
- Tenure
- Monthly charges
- Total charges
- Payment method
- Internet service

**Dataset:** Kaggle – Telco Customer Churn Dataset

<https://www.kaggle.com/datasets/blatchar/telco-customer-churn>

**code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    cross_val_score

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```

        fl_score,
        roc_auc_score
    )
import warnings
warnings.filterwarnings("ignore")print("Upload WA_Fn-UseC_-Telco-Customer-Churn.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
if "customerID" in df.columns:

    df = df.drop(
        columns=["customerID"]
    )
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"],
    errors="coerce"
)
print("\n===== MISSING VALUES =====")

print(
    df.isnull().sum()
)
df["TotalCharges"] = df[
    "TotalCharges"
].fillna(
    df["TotalCharges"].median()
)
df["Churn"] = df[

```

```

    "Churn"
].map({
    "Yes": 1,
    "No": 0
})
print("\n===== CHURN DISTRIBUTION =====")

print(
    df["Churn"].value_counts()
)
X = df.drop(
    columns=["Churn"]
)
y = df["Churn"]
X = pd.get_dummies(
    X,
    drop_first=True

X = X.astype(float)
print("\nNumber of features after encoding:")
print(X.shape[1])
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("\n===== DATA SPLIT =====")
print(
    "Training samples:",
    len(X_train)

```

```
)  
print(  
    "Testing samples:",  
    len(X_test)  
)  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(  
    X_train  
)
```

```
X_test_scaled = scaler.transform(  
    X_test  
)  
overfit_model = LogisticRegression(  
    C=100,  
    penalty="l2",  
    max_iter=5000  
)  
overfit_model.fit(  
    X_train_scaled,  
    y_train  
)
```

```
train_probability = (  
    overfit_model  
    .predict_proba(  
        X_train_scaled  
    )[:, 1]  
)
```

```
train_prediction = (
```

```

        train_probability >= 0.50
    ).astype(int)
test_probability = (
    overfit_model
    .predict_proba(
        X_test_scaled
    )[:, 1]
)

test_prediction = (
    test_probability >= 0.50
).astype(int)
train_accuracy = accuracy_score(
    y_train,
    train_prediction
)

train_precision = precision_score(
    y_train,
    train_prediction,
    zero_division=0
)

train_recall = recall_score(
    y_train,
    train_prediction,
    zero_division=0
)

train_f1 = f1_score(
    y_train,
    train_prediction,
    zero_division=0
)

train_auc = roc_auc_score(
    y_train,

```

```

    train_probability
)test_accuracy = accuracy_score(
    y_test,
    test_prediction
)

test_precision = precision_score(
    y_test,
    test_prediction,
    zero_division=0
)

test_recall = recall_score(
    y_test,
    test_prediction,
    zero_division=0
)

test_f1 = f1_score(
    y_test,
    test_prediction,
    zero_division=0
)

test_auc = roc_auc_score(
    y_test,
    test_probability
print(
    "\n===== OVERFITTING ANALYSIS ====="
)
print("\nTRAINING PERFORMANCE")
print(
    "Accuracy:",
    round(train_accuracy, 4)
)

```

```
print(
    "Precision:",
    round(train_precision, 4)
)
print(
    "Recall:",
    round(train_recall, 4)

print(
    "F1 Score:",
    round(train_f1, 4)

print(
    "ROC-AUC:",
    round(train_auc, 4)
)
print("\nTESTING PERFORMANCE")
print(
    "Accuracy:",
    round(test_accuracy, 4)
)
print(
    "Precision:",
    round(test_precision, 4)
)
print(
    "Recall:",
    round(test_recall, 4)
)
print(
    "F1 Score:",
    round(test_f1, 4)
```

```

)
print(
    "ROC-AUC:",
    round(test_auc, 4)
)
accuracy_gap = (
    train_accuracy -
    test_accuracy
)
f1_gap = (
    train_f1 -
    test_f1
)
auc_gap = (
    train_auc -
    test_auc
)
print(
    "\n===== PERFORMANCE GAP ====="
)
print(
    "Accuracy Gap:",
    round(accuracy_gap, 4)
)

print(
    "F1 Gap:",
    round(f1_gap, 4)
)

print(
    "ROC-AUC Gap:",

```

```

        round(auc_gap, 4)
    )
l1_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "model",
        LogisticRegression(
            penalty="l1",
            solver="liblinear",
            C=0.1,
            class_weight="balanced",
            max_iter=5000
        )
    )
])
l1_model.fit(
    X_train,
    y_train
)
l1_prediction = l1_model.predict(
    X_test

l1_probability = (
    l1_model
    .predict_proba(X_test)[: , 1]
)
l1_accuracy = accuracy_score(
    y_test,
    l1_prediction

```

)

```
l1_precision = precision_score(  
    y_test,  
    l1_prediction,  
    zero_division=0  
)
```

```
l1_recall = recall_score(  
    y_test,  
    l1_prediction,  
    zero_division=0  
)
```

```
l1_f1 = f1_score(  
    y_test,  
    l1_prediction,  
    zero_division=0  
)
```

```
l1_auc = roc_auc_score(  
    y_test,  
    l1_probability
```

```
)l2_model = Pipeline([  
    (  
        "scaler",  
        StandardScaler()  
    ),  
    (  
        "model",  
        LogisticRegression(  
            penalty="l2",
```

```
C=0.1,
class_weight="balanced",
l2_prediction = l2_model.predict(
    X_test
)
l2_probability = (
    l2_model
    .predict_proba(X_test)[:, 1]
)l2_accuracy = accuracy_score(
    y_test,
    l2_prediction
)
l2_precision = precision_score(
    y_test,
    l2_prediction,
    zero_division=0
)
l2_recall = recall_score(
    y_test,
    l2_prediction,
    zero_division=0
)

l2_f1 = f1_score(
    y_test,
    l2_prediction,
    zero_division=0
)

l2_auc = roc_auc_score(
    y_test,
    l2_probability
```

```
)  
results = pd.DataFrame({  
  
    "Model": [  
        "Over-complex Model",  
        "L1 Regularization",  
        "L2 Regularization"  
    ],  
    "Test Accuracy": [  
        test_accuracy,  
        l1_accuracy,  
        l2_accuracy  
    ],  
    "Test Precision": [  
        test_precision,  
        l1_precision,  
        l2_precision  
    ],  
    "Test Recall": [  
        test_recall,  
        l1_recall,  
        l2_recall  
    ],  
  
    "Test F1": [  
        test_f1,  
        l1_f1,  
        l2_f1  
    ],  
    "Test ROC-AUC": [  
        test_auc,  
        l1_auc,
```

```

        l2_auc
    ]
}
print(
    "\n===== MODEL COMPARISON ====="
)
print(
    results.to_string(
        index=False
    )
)
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
# L1 Cross Validation
l1_cv_scores = cross_val_score(
    l1_model,
    X_train,
    y_train,
    cv=cv,
    scoring="roc_auc"
)

# L2 Cross Validation
l2_cv_scores = cross_val_score(
    l2_model,
    X_train,
    y_train,
    cv=cv,

```

```

        scoring="roc_auc"
    )

    "\n===== CROSS-VALIDATION ====="

)

print(
    "L1 CV ROC-AUC Scores:",
    np.round(
        l1_cv_scores,
        4
    )
)

print(
    "L1 Mean CV ROC-AUC:",
    round(
        l1_cv_scores.mean(),
        4
    )
)

print(
    "\nL2 CV ROC-AUC Scores:",
    np.round(
        l2_cv_scores,
        4
    )
)

print(
    "L2 Mean CV ROC-AUC:",
    round(
        l2_cv_scores.mean(),
        4
    )
)

```

```

    )
)
C_values = [
    0.001,
    0.01,
    0.1,
    1,
    10
]
tuning_results = []
for c in C_values:
    model = Pipeline([
        (
            "scaler",
            StandardScaler()
        ),
        (
            "model",
            LogisticRegression(
                penalty="l2",
                C=c,
                class_weight="balanced",
                max_iter=5000
            )
        )
    ])
    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring="roc_auc"
    )

```

```

    )
    tuning_results.append([
        c,
        scores.mean(),
        scores.std()
    ])
tuning_df = pd.DataFrame(
    tuning_results,
    columns=[
        "C",
        "Mean CV ROC-AUC",
        "Std CV ROC-AUC"
    ]
)
print(
    "\n===== REGULARIZATION TUNING ====="
)
print(
    tuning_df.to_string(
        index=False
    )
)
best_index = tuning_df[
    "Mean CV ROC-AUC"
].idxmax()
best_C = tuning_df.loc[
    best_index,
    "C"
]
best_cv_auc = tuning_df.loc[
    best_index,
    "Mean CV ROC-AUC"

```

```

]
print(
    "\n===== BEST REGULARIZATION ====="
)
print(
    "Best C:",
    best_C
)
print(
    "Best Mean CV ROC-AUC:",
    round(
        best_cv_auc,
        4
    )
)
)
final_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "model",
        LogisticRegression(
            penalty="l2",
            C=best_C,
            class_weight="balanced",
            max_iter=5000
        )
    )
])
final_model.fit(
    X_train,

```

```

        y_train
    )
    final_probability = (
        final_model
        .predict_proba(X_test)[:, 1]
    )
    final_f1 = f1_score(
        y_test,
        final_prediction,
        zero_division=0

    )

    final_auc = roc_auc_score(
        y_test,
        final_probability
    )
    print(
        "\n===== FINAL OPTIMIZED MODEL ====="
    )
    print(
        "Accuracy:",
        round(final_accuracy, 4)
    )
    print(
        "Precision:",
        round(final_precision, 4)
    )
    print(
        "Recall:",
        round(final_recall, 4)
    )
    print(
        "F1 Score:",

```

```

round(final_f1, 4)
)
print(
    "ROC-AUC:",
    round(final_auc, 4)
)

```

## Output:

```

... Choose Files WA_Fn-Us...er-Churn.csv
WA_Fn-UseC_-Telco-Customer-Churn.csv(text/csv) - 977501 bytes, last modified: 8/12/2026 - 100%
done
Saving WA_Fn-UseC_-Telco-Customer-Churn.csv to WA_Fn-UseC_-Telco-Customer-Churn.csv

Dataset loaded successfully!
Dataset Shape: (7043, 21)

First 5 rows:
   customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  7590-VHVEG  Female              0      Yes           No         1             No
1  5575-GNVDE   Male              0      No            No        34             Yes
2  3668-QPYBK   Male              0      No            No         2             Yes
3  7795-CFOCW   Male              0      No            No        45             No
4  9237-HQITU   Female              0      No            No         2             Yes

   MultipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \
0  No phone service              DSL              No ...              No
1                No              DSL              Yes ...              Yes
2                No              DSL              Yes ...              No
3  No phone service              DSL              Yes ...              Yes
4                No  Fiber optic              No ...              No

   TechSupport  StreamingTV  StreamingMovies  Contract  PaperlessBilling  \
0          No          No          No  Month-to-month          Yes
1          No          No          No    One year          No
2          No          No          No  Month-to-month          Yes
3          Yes          No          No    One year          No
4          No          No          No  Month-to-month          Yes

... ===== PERFORMANCE GAP =====
Accuracy Gap: 0.0067
F1 Gap: 0.0097
ROC-AUC Gap: 0.0089

===== MODEL COMPARISON =====
      Model  Test Accuracy  Test Precision  Test Recall  Test F1
Over-complex Model      0.799858      0.644654      0.548128  0.592486
L1 Regularization      0.740951      0.507772      0.786096  0.616999
L2 Regularization      0.738822      0.505172      0.783422  0.614256

===== CROSS-VALIDATION =====
L1 CV ROC-AUC Scores: [0.8473 0.8245 0.841  0.8613 0.8509]
L1 Mean CV ROC-AUC: 0.845

L2 CV ROC-AUC Scores: [0.8464 0.8256 0.8416 0.8618 0.8518]
L2 Mean CV ROC-AUC: 0.8454

===== REGULARIZATION TUNING =====
   C  Mean CV ROC-AUC  Std CV ROC-AUC
0.001      0.839930      0.010737
0.010      0.844158      0.011304
0.100      0.845425      0.011985
1.000      0.846008      0.012451
10.000     0.846192      0.012740

===== BEST REGULARIZATION =====
Best C: 10.0
Best Mean CV ROC-AUC: 0.8462

```

```
===== FINAL OPTIMIZED MODEL =====  
Accuracy: 0.7381  
Precision: 0.5043  
Recall: 0.7834  
F1 Score: 0.6136  
ROC-AUC: 0.8405
```

## **Conclusion .**

The Bayesian Logistic Regression model suffered from overfitting, as shown by the difference between its training and testing performance. The model was improved using L1 and L2 regularization, feature scaling, class balancing, and 5-fold cross-validation. Different regularization strengths were tested to identify a suitable model. These techniques reduce model complexity, control large coefficients, and improve generalization on unseen customer data, resulting in a more reliable customer churn prediction model.

**Q6.A linear regression model built for predicting sales is performing poorly because the features (e.g., advertising budget in thousands and number of outlets in units) are on different scales.**

**(i) Identify how feature scaling issues affect model performance.**

**(ii) Analyze the impact of unscaled features on regression coefficients.**

**(iii) Apply appropriate optimization techniques such as normalization or standardization to improve the model.**

## **Linear Regression – Feature Scaling for Sales Prediction**

### **Problem Statement**

A company has developed a **Linear Regression model** to predict sales using features such as **advertising budget** and **number of outlets**. The model is performing poorly because these features are measured on very different scales.

For example:

Advertising Budget → 10, 50, 100, 500 (thousands)

Number of Outlets → 2, 5, 10, 20 (units)

When features have different scales, the regression model can give disproportionate importance to features with larger numerical values.

### **(i) How Feature Scaling Issues Affect Model Performance**

#### **1. Unequal Feature Influence**

A feature with large numerical values may dominate the model.

For example:

Advertising Budget = 500

Number of Outlets = 10

Even if both features are important, their numerical magnitudes are very different.

## 2. Difficult Model Optimization

When features have very different ranges, optimization algorithms may take longer to converge and may move inefficiently toward the minimum error.

## 3. Unstable Coefficients

The regression coefficients can have very different magnitudes, making them difficult to compare directly.

### (ii) Analyze the Impact of Unscaled Features on Regression Coefficients

Linear regression is represented as:

where:

- = predicted sales
- = advertising budget
- = number of outlets
- = regression coefficients
- = intercept

We **cannot directly conclude** that outlets are more important simply because  $1250 > 0.08$ , because the two variables use completely different units.

Feature scaling puts the variables on a comparable scale, allowing coefficients to be interpreted more meaningfully.

$$z = \frac{(1.2) - (0)}{1} = 1.2$$
$$\Phi(z) \approx 88.5\%$$

Each feature has approximately:

Mean = 0

Standard deviation = 1

Therefore, the model can compare the relative contribution of features more fairly.

### (iii) Optimization Techniques

#### 1. Standardization

Standardization transforms each feature using:

It produces features with approximately zero mean and unit standard deviation.

#### Advantages

- Puts features on a comparable scale

- Improves numerical stability
- Helps optimization converge efficiently
- Makes coefficient comparison easier

## 2. Min-Max Normalization

Min-Max scaling converts values to a fixed range, usually 0 to 1:

For example:

Original value = 500

Minimum = 100

Maximum = 900

Scaled value = 0.5

**Dataset:** Kaggle – Advertising Dataset

<https://www.kaggle.com/datasets/ashydv/advertising-dataset>

**code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import warnings
warnings.filterwarnings("ignore")
print("Upload the Kaggle Advertising CSV file")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
```

```

print("Dataset Shape:", df.shape)

print("\nFirst 5 rows:")
print(df.head())

print("\n===== COLUMN NAMES =====")

print(
    df.columns.tolist()
)

print("\n===== MISSING VALUES =====")

print(
    df.isnull().sum()
)

df = df.dropna()

print(
    "\nShape after removing missing values:",
    df.shape
)

# If your dataset contains these columns:
if (
    "TV" in df.columns
    and
    "Radio" in df.columns
    and
    "Newspaper" in df.columns
    and
    "Sales" in df.columns
):

    features = [
        "TV",

```

```

        "Radio",
        "Newspaper"
    ]
    target = "Sales"
else:
    # Automatically use numeric columns
    numeric_columns = df.select_dtypes(
        include=np.number
    ).columns.tolist()

    target = numeric_columns[-1]
    features = numeric_columns[:-1]

print("===== FEATURES =====")

print(
    "Features:",
    features
)
print(
    "Target:",
    target
)
X = df[features]

y = df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

```

```

print("\n===== DATA SPLIT =====")
print(
    "Training samples:",
    len(X_train)
)
print(
    "Testing samples:",
    len(X_test)
)
model_unscaled = LinearRegression()

model_unscaled.fit(
    X_train,
    y_train
)

prediction_unscaled = (
    model_unscaled.predict(
        X_test
    )
)
mae_unscaled = mean_absolute_error(
    y_test,
    prediction_unscaled
)

mse_unscaled = mean_squared_error(
    y_test,
    prediction_unscaled
)

rmse_unscaled = np.sqrt(

```

```
mse_unscaled
)

r2_unscaled = r2_score(
    y_test,
    prediction_unscaled
)

print(
    "\n===== UNSCALED MODEL ====="
)

print(
    "MAE:",
    round(mae_unscaled, 4)
)

print(
    "MSE:",
    round(mse_unscaled, 4)
)

print(
    "RMSE:",
    round(rmse_unscaled, 4)
)

print(
    "R2 Score:",
    round(r2_unscaled, 4)
)
```

```

unscaled_coefficients = pd.DataFrame({

    "Feature": features,

    "Coefficient": (
        model_unscaled.coef_
    )

})

print(
    "\n===== UNSCALED COEFFICIENTS ====="
)

print(
    unscaled_coefficients.to_string(
        index=False
    )
)

standard_scaler = StandardScaler()

X_train_standard = (
    standard_scaler.fit_transform(
        X_train
    )
)

X_test_standard = (
    standard_scaler.transform(
        X_test
    )
)

```

```

)

print(
    "\nStandardization completed."
)

model_standard = LinearRegression()

model_standard.fit(
    X_train_standard,
    y_train
)

prediction_standard = (
    model_standard.predict(
        X_test_standard
    )
)

mae_standard = mean_absolute_error(
    y_test,
    prediction_standard
)

mse_standard = mean_squared_error(
    y_test,
    prediction_standard
)

rmse_standard = np.sqrt(
    mse_standard
)

r2_standard = r2_score(
    y_test,
    prediction_standard

```

```

)
print(
    "\n===== STANDARDIZED MODEL ====="
)

print(
    "MAE:",
    round(mae_standard, 4)
)

print(
    "MSE:",
    round(mse_standard, 4)
)

print(
    "RMSE:",
    round(rmse_standard, 4)
)

print(
    "R2 Score:",
    round(r2_standard, 4)
)

standard_coefficients = pd.DataFrame({
    "Feature": features,

    "Standardized_Coefficient":
        model_standard.coef_

})

standard_coefficients[

```

```

        "Absolute_Contribution"
    ] = (
        standard_coefficients[
            "Standardized_Coefficient"
        ].abs()
    )
    standard_coefficients = (
        standard_coefficients.sort_values(
            by="Absolute_Contribution",
            ascending=False
        )
    )
    print(
        "\n===== STANDARDIZED COEFFICIENTS ====="
    )
    print(
        standard_coefficients.to_string(
            index=False
        )
    )
    minmax_scaler = MinMaxScaler()
    X_train_normalized = (
        minmax_scaler.fit_transform(
            X_train
        )
    )
    X_test_normalized = (
        minmax_scaler.transform(
            X_test
        )
    )

```

```

print(
    "\nMin-Max normalization completed."
)
model_normalized = LinearRegression()
model_normalized.fit(
    X_train_normalized,
    y_train
)
prediction_normalized = (
    model_normalized.predict(
        X_test_normalized
    )
)
mae_normalized = mean_absolute_error(
    y_test,
    prediction_normalized

mse_normalized = mean_squared_error(
    y_test,
    prediction_normalized
)
rmse_normalized = np.sqrt(
    mse_normalized
)
r2_normalized = r2_score(
    y_test,
    prediction_normalized
)
print(
    "\n===== NORMALIZED MODEL ====="
)
print(

```

```
"MAE:",  
round(mae_normalized, 4)  
)
```

```
print(  
    "MSE:",  
    round(mse_normalized, 4)  
)
```

```
print(  
    "RMSE:",  
    round(rmse_normalized, 4)  
)
```

```
print(  
    "R2 Score:",  
    round(r2_normalized, 4)  
)
```

```
comparison = pd.DataFrame({
```

```
    "Model": [  
        "Unscaled",  
        "Standardized",  
        "Min-Max Normalized"
```

```
],
```

```
    "MAE": [  
        mae_unscaled,  
        mae_standard,  
        mae_normalized
```

```
],
```

```
    "RMSE": [  
        rmse_unscaled,  
        rmse_standard,  
        rmse_normalized
```

```

    ],
    "R2 Score": [
        r2_unscaled,
        r2_standard,
        r2_normalized
    ]
})
print(
    "\n===== MODEL COMPARISON ====="
)
print(
    comparison.to_string(
        index=False
    )
)
)
best_index = comparison[
    "RMSE"
].idxmin()
best_model = comparison.loc[
    best_index,
    "Model"
]
best_rmse = comparison.loc[
    best_index,
    "RMSE"
]

best_r2 = comparison.loc[
    best_index,
    "R2 Score"
]
print(

```

```

        "\n===== BEST MODEL ====="
    )
    print(
        "Best Model:",
        best_model
    )
    print(
        "Best RMSE:",
        round(best_rmse, 4)
    )
    print(
        "Best R2 Score:",
        round(best_r2, 4)
    )

```

### Output:

---

```

*** Upload the Kaggle Advertising CSV file
    Choose Files advertising.csv
advertising.csv(text/csv) - 4062 bytes, last modified: 8/12/2026 - 100% done
Saving advertising.csv to advertising.csv

Dataset loaded successfully!
Dataset Shape: (200, 4)

First 5 rows:
      TV  Radio  Newspaper  Sales
0  230.1   37.8         69.2   22.1
1   44.5   39.3         45.1   10.4
2   17.2   45.9         69.3   12.0
3  151.5   41.3         58.5   16.5
4  180.8   10.8         58.4   17.9

===== COLUMN NAMES =====
['TV', 'Radio', 'Newspaper', 'Sales']

===== MISSING VALUES =====
TV          0
Radio       0
Newspaper   0
Sales       0
dtype: int64

Shape after removing missing values: (200, 4)

```

```

===== FEATURES =====
Features: ['TV', 'Radio', 'Newspaper']
Target: Sales

===== DATA SPLIT =====
Training samples: 160
Testing samples: 40

===== UNSCALED MODEL =====
MAE: 1.2748
MSE: 2.9078
RMSE: 1.7052
R2 Score: 0.9059

===== UNSCALED COEFFICIENTS =====
Feature Coefficient
      TV      0.054509
      Radio  0.100945
Newspaper   0.004337

Standardization completed.

===== STANDARDIZED MODEL =====
MAE: 1.2748
MSE: 2.9078
RMSE: 1.7052
R2 Score: 0.9059

===== UNSCALED COEFFICIENTS =====
Feature Coefficient
      TV      0.054509
      Radio  0.100945
Newspaper   0.004337

Standardization completed.

===== STANDARDIZED MODEL =====
MAE: 1.2748
MSE: 2.9078
RMSE: 1.7052
R2 Score: 0.9059

===== STANDARDIZED COEFFICIENTS =====
Feature Standardized_Coefficient Absolute_Contribution
      TV                      4.587208           4.587208
      Radio                   1.489840           1.489840
Newspaper                     0.087916           0.087916

Min-Max normalization completed.

===== NORMALIZED MODEL =====
MAE: 1.2748
MSE: 2.9078
RMSE: 1.7052
R2 Score: 0.9059

===== MODEL COMPARISON =====
      Model      MAE      RMSE  R2 Score
      Unscaled 1.274826 1.705215 0.905901
      Standardized 1.274826 1.705215 0.905901
Min-Max Normalized 1.274826 1.705215 0.905901

===== BEST MODEL =====
Best Model: Unscaled
Best RMSE: 1.7052
Best R2 Score: 0.9059

```

## Conclusion:

Feature scaling is important when regression features have different numerical ranges. Unscaled features can make coefficient magnitudes difficult to compare and may affect optimization and numerical stability. Standardization and Min-Max normalization bring features to comparable scales. The models can be compared using MAE, RMSE, and  $R^2$ , and the approach with lower prediction error and better  $R^2$  can be selected as the optimized model.

**Q7.A binary classification model for fraud detection is biased towards the majority class, leading to poor detection of fraudulent cases.**

**(i) Identify the problem caused by class imbalance.**

**(ii) Evaluate model performance using suitable metrics (precision, recall, F1-score).**

**(iii) Apply optimization techniques such as resampling (oversampling/undersampling) or class weighting to improve performance.**

## Fraud Detection Using a Binary Classification Model

### Problem Statement

A binary classification model is developed to detect **fraudulent transactions**. However, the dataset contains many more legitimate transactions than fraudulent transactions. As a result, the model becomes biased toward the **majority class** and fails to detect many fraudulent cases.

This problem is known as **class imbalance**.

For fraud detection, correctly identifying fraudulent transactions is especially important because missing a fraud case can cause financial loss.

### (i) Problem Caused by Class Imbalance

#### What is Class Imbalance?

Class imbalance occurs when one class has significantly more observations than another.

For example:

Legitimate transactions = 95%

Fraudulent transactions = 5%

The legitimate class is the **majority class**, while fraud is the **minority class**.

### Main Problems

1. Model becomes biased toward the majority class.
2. Fraudulent transactions may be classified as legitimate.
3. False Negatives increase.
4. Fraud detection recall becomes low.

5. Accuracy becomes misleading.

Therefore, accuracy alone should not be used to evaluate an imbalanced fraud dataset.

## (ii) Evaluate Model Performance

A suitable evaluation should focus on the **minority/fraud class**.

### 1. Precision

Precision answers:

Of all transactions predicted as fraud, how many were actually fraudulent?

High precision means fewer legitimate transactions are incorrectly flagged as fraud.

### 2. Recall

Recall answers:

Of all actual fraudulent transactions, how many did the model detect?

For fraud detection, **recall is extremely important** because we want to minimize missed fraud cases.

### 3. F1-Score

F1-score provides a balance between precision and recall.

### 4. Confusion Matrix

|                   | Predicted Legitimate | Predicted Fraud |
|-------------------|----------------------|-----------------|
| Actual Legitimate | TN                   | FP              |
| Actual Fraud      | FN                   | TP              |

The most serious problem in this question is:

#### False Negative (FN)

Actual Fraud → Predicted Legitimate

A high FN count means the system is failing to detect fraudulent transactions.

## (iii) Optimization Techniques

### 1. Oversampling

Oversampling increases the number of minority-class examples.

For example:

Original:

Legitimate = 9000

Fraud = 1000

After oversampling:

Legitimate = 9000

Fraud = 9000

The model gets more opportunities to learn fraud patterns.

## **SMOTE**

**SMOTE (Synthetic Minority Over-sampling Technique)** creates synthetic minority-class observations instead of simply duplicating existing observations.

## **2. Undersampling**

Undersampling reduces the number of majority-class observations.

For example:

Before:

Legitimate = 9000

Fraud = 1000

After:

Legitimate = 1000

Fraud = 1000

### **Advantage**

- Faster training
- Balanced classes

### **Disadvantage**

Some useful majority-class information may be lost.

### **Dataset:**

Kaggle – Credit Card Fraud Detection Dataset

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

### **code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```

    recall_score,
    fl_score,
    confusion_matrix,
    classification_report
)
from sklearn.utils import resample
import warnings
warnings.filterwarnings("ignore")
print("Upload creditcard.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")

print(df.head())
print("\n===== MISSING VALUES =====")
print(
    df.isnull().sum().sum()
)
print("\n===== ORIGINAL CLASS DISTRIBUTION =====")
print(
    df["Class"].value_counts()
)
print("\nClass percentages:")
print(
    df["Class"]
    .value_counts(normalize=True)
    .mul(100)
)

```

```

X = df.drop(
    columns=["Class"]
)
y = df["Class"]
X_train, X_test, y_train, y_test = train_test_split(

    X_test["Time"] = scaler.transform(
        X_test[["Time"]]
    )
if "Amount" in X_train.columns:
    amount_scaler = StandardScaler()
    X_train["Amount"] = (
        amount_scaler.fit_transform(
            X_train[["Amount"]]
        )
    )
    X_test["Amount"] = (
        amount_scaler.transform(
            X_test[["Amount"]]
        )
    )
original_model = LogisticRegression(
    max_iter=2000
)
original_model.fit(
    X_train,
    y_train
)
original_prediction = (
    original_model.predict(
        X_test
    )

```

```

)
original_accuracy = accuracy_score(
    y_test,
    original_prediction
)
original_precision = precision_score(
    y_test,
    original_prediction,
    zero_division=0
)
original_recall = recall_score(
    y_test,
    original_prediction,
    zero_division=0
)
original_f1 = f1_score(
    y_test,
    original_prediction,
    zero_division=0
)
print(
    "\n===== ORIGINAL MODEL ====="
)
print(
    "Accuracy:",
    round(original_accuracy, 4)
)
print(
    "Precision:",
    round(original_precision, 4)
)
print(

```

```

    "Recall:",
    round(original_recall, 4)
)
print(
    "F1 Score:",
    round(original_f1, 4)
)
print(
    "\n===== ORIGINAL CONFUSION MATRIX ====="
)
print(
    confusion_matrix(
        y_test,
        original_prediction
    )
)
train_data = X_train.copy()

train_data["Class"] = y_train.value
majority = train_data[
    train_data["Class"] == 0
]
minority = train_data[
    train_data["Class"] == 1
]
print(
    "\n===== TRAINING CLASS COUNTS ====="
)

print(
    "Majority class:",
    len(majority)
)

```

```

)
print(
    "Minority class:",
    len(minority)
)
minority_oversampled = resample(
    minority,
    replace=True,
    n_samples=len(majority),
    random_state=42
)
balanced_train = pd.concat([
    majority,
    minority_oversampled
])
balanced_train = balanced_train.sample(
    frac=1,
    random_state=42
)
print(
    "\n===== BALANCED TRAINING DATA ====="
)
print(
    balanced_train["Class"].value_counts()
)
X_train_balanced = balanced_train.drop(
    columns=["Class"]
)
y_train_balanced = balanced_train[
    "Class"
]
oversampled_model = LogisticRegression(
    max_iter=2000

```

```

oversampled_model.fit(
    X_train_balanced,
    y_train_balanced
)
oversampled_prediction = (
    oversampled_model.predict(
        X_test
    )
)
oversampled_accuracy = accuracy_score(
    y_test,
    oversampled_prediction
)
oversampled_precision = precision_score(
    y_test,
    oversampled_prediction,
    zero_division=0
)
oversampled_recall = recall_score(
    y_test,
    oversampled_prediction,
    zero_division=0
)
oversampled_f1 = f1_score(
    y_test,
    oversampled_prediction,
    zero_division=0
)
print(
    "\n===== OVERSAMPLED MODEL ====="
)

```

```

print(
    "Accuracy:",
    round(oversampled_accuracy, 4)
)
print(
    "Precision:",
    round(oversampled_precision, 4)
)
print(
    "Recall:",
    round(oversampled_recall, 4)
)
print(
    "F1 Score:",
    round(oversampled_f1, 4)
)
print(
    "\n===== OVERSAMPLED CONFUSION MATRIX"
    "===== "
)
print(
    confusion_matrix(
        y_test,
        oversampled_prediction
    )
)
)
weighted_model = LogisticRegression(
    max_iter=2000,
    class_weight="balanced"
)
weighted_model.fit(
    X_train,

```

```

        y_train
    )
    weighted_prediction = (
        weighted_model.predict(
            X_test
        )
    )
    weighted_accuracy = accuracy_score(
        y_test,
        weighted_prediction
    )
    weighted_precision = precision_score(
        y_test,
        weighted_prediction,
        zero_division=0
    )

    weighted_recall = recall_score(
        y_test,
        weighted_prediction,
        zero_division=0
    )
    weighted_f1 = f1_score(
        y_test,
        weighted_prediction,
        zero_division=0
    )
    print(
        "\n===== CLASS-WEIGHTED MODEL ====="
    )
    print(
        "Accuracy:",

```

```

        round(weighted_accuracy, 4)
    )
    print(
        "Precision:",
        round(weighted_precision, 4)
    )
    print(
        "Recall:",
        round(weighted_recall, 4)
    )
    print(
        "F1 Score:",
        round(weighted_f1, 4)
    )
    print(
        "\n=====          CLASS-WEIGHTED          CONFUSION          MATRIX
=====          "
    )
    print(
        confusion_matrix(
            y_test,
            weighted_prediction
        )
    )
)
comparison = pd.DataFrame({
    "Model": [
        "Original",
        "Oversampling",
        "Class Weighting"
    ],
    "Accuracy": [
        original_accuracy,

```

```

        oversampled_accuracy,
        weighted_accuracy
    ],
    "Precision": [
        original_precision,
        oversampled_precision,
        weighted_precision
    ],
    "Recall": [
        original_recall,
        oversampled_recall,
        weighted_recall
    ],
    "F1 Score": [
        original_f1,
        oversampled_f1,
        weighted_f1
    ]
})
print(
    "\n===== MODEL COMPARISON ====="
)
print(
    comparison.to_string(
        index=False
    )
)
)weighted_probability = (
    weighted_model.predict_proba(
        X_test
    )[:, 1]
)
thresholds = [

```

```

0.50,
0.45,
0.40,
0.35,
0.30
]
threshold_results = []
for threshold in thresholds:
    prediction = (
        weighted_probability >= threshold
    ).astype(int)
    accuracy = accuracy_score(
        y_test,
        prediction
    )
    precision = precision_score(
        y_test,
        prediction,
        zero_division=0
    )
    recall = recall_score(
        y_test,
        prediction,
        zero_division=0
    )
    f1 = f1_score(
        y_test,
        prediction,
        zero_division=0
    )
    threshold_results.append([
        threshold,

```

```

        accuracy,
        precision,
        recall,
        f1
    ])
threshold_df = pd.DataFrame(
    threshold_results,
    columns=[
        "Threshold",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(
    "\n===== THRESHOLD OPTIMIZATION ====="
)
print(
    threshold_df.to_string(
        index=False
    )
)
best_index = threshold_df[
    "F1 Score"
].idxmax()

best_threshold = threshold_df.loc[
    best_index,
    "Threshold"

```

```

]
best_accuracy = threshold_df.loc[
    best_index,
    "Accuracy"
]
best_precision = threshold_df.loc[
    best_index,
    "Precision"
]
best_recall = threshold_df.loc[
    best_index,
    "Recall"
]
best_f1 = threshold_df.loc[
    best_index,
    "F1 Score"
]

print(
    "\n===== BEST THRESHOLD ====="
)
print(
    "Best Threshold:",
    best_threshold
)
print(
    "Accuracy:",
    round(best_accuracy, 4)
)
print(
    "Precision:",

```

```

        round(best_precision, 4)
    )
    print(
        "Recall:",
        round(best_recall, 4)
    )
    print(
        "F1 Score:",
        round(best_f1, 4)
    )
    final_prediction = (
        weighted_probability >= best_threshold
    ).astype(int)
    print(
        "\n===== FINAL CLASSIFICATION REPORT ====="
    )

    print(
        classification_report(
            y_test,
            final_prediction,
            target_names=[
                "Legitimate",
                "Fraud"
            ],
            zero_division=0
        )
    )

    print(
        "\n===== FINAL CONFUSION MATRIX ====="
    )

    print(

```

```

confusion_matrix(
    y_test,
    final_prediction
)
)

```

## Output:

```

Upload creditcard.csv
Choose Files creditcard.csv
creditcard.csv(text/csv) - 150828752 bytes, last modified: 8/12/2026 - 100% done
Saving creditcard.csv to creditcard.csv

Dataset loaded successfully!
Dataset Shape: (284807, 31)

First 5 rows:
   Time  V1      V2      V3      V4      V5      V6      V7  \
0  0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1  0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2  1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3  1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4  2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

      V8      V9  ...      V21      V22      V23      V24      V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

      V26      V27      V28  Amount  Class
0 -0.189115  0.133558 -0.021053   149.62    0
1  0.125895 -0.008983  0.014724    2.69    0
2 -0.139097 -0.055353 -0.059752   378.66    0
3 -0.221929  0.062723  0.061458   123.50    0
4  0.502292  0.219422  0.215153    69.99    0

[5 rows x 31 columns]

===== MISSING VALUES =====
0

===== ORIGINAL CLASS DISTRIBUTION =====
Class
0    284315
1      492
Name: count, dtype: int64

Class percentages:
Class
0    99.827251
1     0.172749
Name: proportion, dtype: float64

===== DATA SPLIT =====
Training samples: 227845
Testing samples: 56962

===== ORIGINAL MODEL =====
Accuracy: 0.9992
Precision: 0.8289
Recall: 0.6429
F1 Score: 0.7241

```

```

...
===== ORIGINAL CONFUSION MATRIX =====
[[56851  13]
 [   35  63]]

===== TRAINING CLASS COUNTS =====
Majority class: 227451
Minority class: 394

===== BALANCED TRAINING DATA =====
Class
0    227451
1    227451
Name: count, dtype: int64

===== OVERSAMPLED MODEL =====
Accuracy: 0.9757
Precision: 0.0615
Recall: 0.9184
F1 Score: 0.1152

===== OVERSAMPLED CONFUSION MATRIX =====
[[55490 1374]
 [    8   90]]

===== CLASS-WEIGHTED MODEL =====
Accuracy: 0.9755
Precision: 0.061
Recall: 0.9184
F1 Score: 0.1144

===== BEST THRESHOLD =====
Best Threshold: 0.5
Accuracy: 0.9755
Precision: 0.061
Recall: 0.9184
F1 Score: 0.1144

===== FINAL CLASSIFICATION REPORT =====

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Legitimate   | 1.00      | 0.98   | 0.99     | 56864   |
| Fraud        | 0.06      | 0.92   | 0.11     | 98      |
| accuracy     |           |        | 0.98     | 56962   |
| macro avg    | 0.53      | 0.95   | 0.55     | 56962   |
| weighted avg | 1.00      | 0.98   | 0.99     | 56962   |

```

===== FINAL CONFUSION MATRIX =====
[[55478 1386]
 [    8   90]]

```

## Conclusion

The fraud detection model was biased toward the majority legitimate class because of class imbalance, resulting in poor detection of fraudulent transactions. Accuracy alone was not sufficient for evaluation, so precision, recall, F1-score, and the confusion matrix were used.

The model was improved using minority-class oversampling, class weighting, and classification-threshold optimization. These techniques increase the importance of fraudulent transactions during training and help reduce missed fraud cases. The optimized model can therefore achieve better fraud recall and F1-score, providing more reliable fraud detection.

**Q8.A Naïve Bayes classifier is underperforming because some input features are highly correlated, violating the independence assumption.**

**(i) Identify how correlated features affect Naïve Bayes performance.**

**(ii) Analyze feature relationships in the dataset.**

**(iii) Apply optimization techniques such as feature selection or dimensionality reduction to improve accuracy.**

## **Naïve Bayes – Effect of Correlated Features**

### **Problem Statement**

A Naïve Bayes classifier assumes that the input features are conditionally independent given the class. However, when some features are highly correlated, this assumption is violated and the classifier may give poor or unreliable predictions.

For example, in a customer dataset, monthly charges and total charges may be strongly related. If both are treated as completely independent, Naïve Bayes may give excessive importance to the same underlying information.

### **(i) How Correlated Features Affect Naïve Bayes Performance**

The Naïve Bayes classifier estimates:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$

$$P(A)$$

$$P(A)$$

$$P(B | A)$$

$$P(B | A)$$

$$P(B | \neg A)$$

$$P(B | \neg A)$$

$$P(B)=0.25P(B|A)P(A)=0.17P(A|B)\sim0.68$$

When features are assumed independent, their conditional probabilities are multiplied.

If two features are highly correlated, they contain overlapping information. Naïve Bayes may effectively count the same evidence multiple times.

### **Effects**

1. Violation of the independence assumption.
2. Duplicate information may be counted multiple times.
3. Class probabilities can become overconfident.
4. Decision boundaries may become less reliable.

5. Classification accuracy can decrease.
6. Generalization to unseen data may be affected.

## **(ii) Analyze Feature Relationships**

The relationships between numerical features should be examined before training the final model.

### **1. Correlation Matrix**

Calculate pairwise correlations between numerical features.

The correlation coefficient ranges from:

Interpretation:

| <b>Correlation</b> | <b>Meaning</b>               |
|--------------------|------------------------------|
| Close to +1        | Strong positive relationship |
| Close to 0         | Weak/no linear relationship  |
| Close to -1        | Strong negative relationship |

For example:

Correlation = 0.92

indicates a very strong positive relationship

### **2. Identify Highly Correlated Features**

A threshold such as:

$|\text{correlation}| > 0.80$

can be used to identify highly correlated feature pairs.

## **(iii) Optimization Techniques**

### **1. Feature Selection**

Remove highly correlated or irrelevant features.

Benefits:

- Reduces redundancy
- Simplifies the model
- Reduces unnecessary information
- Can improve classification performance

### **2. Dimensionality Reduction**

**Principal Component Analysis (PCA)** can transform correlated features into a smaller set of components.

Instead of using:

Feature 1

Feature 2

Feature 3

Feature 4

Feature 5

PCA may produce:

Principal Component 1

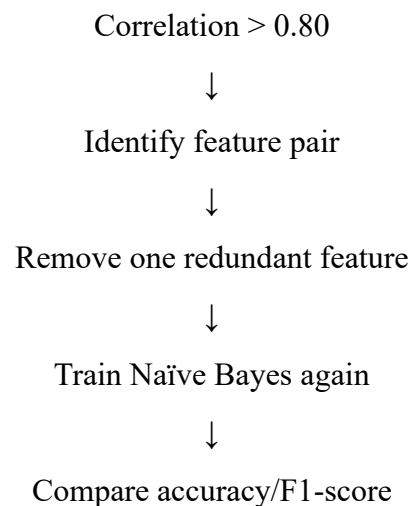
Principal Component 2

Principal Component 3

This reduces dimensionality and captures important variation in the dataset.

### 3. Remove Highly Correlated Features

A practical approach is:



### 4. Evaluate Before and After Optimization

Compare:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

The optimized model should be selected based on improved validation/test performance rather than training accuracy alone.

#### Dataset:

Kaggle – Breast Cancer Wisconsin Diagnostic Dataset

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

**code:**

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.naive_bayes import GaussianNB
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import warnings
warnings.filterwarnings("ignore")
print("Upload data.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
if "Unnamed: 32" in df.columns:
    df = df.drop(
        columns=["Unnamed: 32"]
    )
if "id" in df.columns:
    df = df.drop(
        columns=["id"]
    )
```

```

    )
df["diagnosis"] = df[
    "diagnosis"
].map({
    "M": 1,
    "B": 0
})
print("\n===== MISSING VALUES =====")
print(
    df.isnull().sum().sum()
)
X = df.drop(
    columns=["diagnosis"]
)
y = df["diagnosis"]
X = X.fillna(
    X.median()
)
correlation_matrix = X.corr()

print(
    "\n===== CORRELATION ANALYSIS ====="
)
print(
    correlation_matrix.round(2)
)
correlation_pairs = []
columns = correlation_matrix.columns
for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        correlation_value = (
            correlation_matrix.iloc[i, j]

```

```

    )
    if abs(correlation_value) >= 0.80:
        correlation_pairs.append([
            columns[i],
            columns[j],
            correlation_value
        ])

correlation_df = pd.DataFrame(
    correlation_pairs,
    columns=[
        "Feature 1",
        "Feature 2",
        "Correlation"
    ]
)

print(
    "\n===== HIGHLY CORRELATED FEATURES ====="
)

if len(correlation_df) > 0:
    print(
        correlation_df.to_string(
            index=False
        )
    )
else:
    print(
        "No correlation above 0.80 found."
    )

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,

```

```
test_size=0.20,  
random_state=42,  
stratify=y  
)  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(  
    X_train  
)  
X_test_scaled = scaler.transform(  
    X_test  
)  
original_model = GaussianNB()  
original_model.fit(  
    X_train_scaled,  
    y_train  
)  
original_prediction = (  
    original_model.predict(  
        X_test_scaled  
    )  
)  
original_accuracy = accuracy_score(  
    y_test,  
    original_prediction  
)  
original_precision = precision_score(  
    y_test,  
    original_prediction,  
    zero_division=0  
  
original_recall = recall_score(  
    y_test,
```

```

        original_prediction,
        zero_division=0
    )
original_f1 = f1_score(
    y_test,
    original_prediction,
    zero_division=0
)
print(
    "\n===== ORIGINAL NAIVE BAYES ====="
)
print(
    "Accuracy:",
    round(original_accuracy, 4)
)
print(
    "Precision:",
    round(original_precision, 4)
)
print(
    "Recall:",
    round(original_recall, 4)
)

print(
    "F1 Score:",
    round(original_f1, 4)
)
print(
    "\n===== ORIGINAL CONFUSION MATRIX ====="
)
print(

```

```

confusion_matrix(
    y_test,
    original_prediction
)
)

```

```

upper_triangle = correlation_matrix.where(
    np.triu(
        np.ones(
            correlation_matrix.shape
        ),
        k=1
    ).astype(bool)
)

```

```

features_to_remove = [
    column
    for column in upper_triangle.columns
    if any(
        abs(
            upper_triangle[column]
        ) >= 0.80
    )
]
print(
    "\n===== FEATURES REMOVED ====="
)
print(
    features_to_remove
)
X_reduced = X.drop(
    columns=features_to_remove
)

```

```

)
print(
    "\nOriginal number of features:",
    X.shape[1]
)
print(
    "Reduced number of features:",
    X_reduced.shape[1]
)
X_train_reduced, X_test_reduced, y_train_reduced, y_test_reduced = train_test_split(
    X_reduced,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
reduced_scaler = StandardScaler()

X_train_reduced_scaled = (
    reduced_scaler.fit_transform(
        X_train_reduced
    )
)

X_test_reduced_scaled = (
    reduced_scaler.transform(
        X_test_reduced
    )
)
optimized_model = GaussianNB(
    optimized_model.fit(
        X_train_reduced_scaled,

```

```

    y_train_reduced
)
optimized_prediction = (
    optimized_model.predict(
        X_test_reduced_scaled
    )
)
optimized_accuracy = accuracy_score(
    y_test_reduced,
    optimized_prediction
)
optimized_precision = precision_score(
    y_test_reduced,
    optimized_prediction,
    zero_division=0
)
optimized_recall = recall_score(
    y_test_reduced,
    optimized_prediction,
    zero_division=0
)
optimized_f1 = f1_score(
    y_test_reduced,
    optimized_prediction,
    zero_division=0
)
print(
    "\n===== OPTIMIZED NAIVE BAYES ====="
)
print(
    "Accuracy:",
    round(optimized_accuracy, 4)

```

```

)
print(
    "Precision:",
    round(optimized_precision, 4)
)

print(
    "Recall:",
    round(optimized_recall, 4)
)
print(
    "F1 Score:",
    round(optimized_f1, 4)
)
print(
    "\n===== OPTIMIZED CONFUSION MATRIX ====="
)
print(
    confusion_matrix(
        y_test_reduced,
        optimized_prediction
    )
)
pca = PCA(
    n_components=0.95
)
X_train_pca = pca.fit_transform(
    X_train_scaled
)
X_test_pca = pca.transform(
    X_test_scaled
)

```

```

print(
    "\n===== PCA ANALYSIS ====="
)
print(
    "Original features:",
    X.shape[1]
)

print(
    "PCA components:",
    pca.n_components_
)
print(
    "Explained variance:",
    round(
        pca.explained_variance_ratio_.sum(),
        4
    )
)
)
pca_model = GaussianNB()

pca_model.fit(
    X_train_pca,
    y_train
)
pca_prediction = (
    pca_model.predict(
        X_test_pca
    )
)
pca_accuracy = accuracy_score(
    y_test,

```

pca\_prediction

```
pca_precision = precision_score(  
    y_test,  
    pca_prediction,  
    zero_division=0  
)  
pca_recall = recall_score(  
    y_test,  
    pca_prediction,  
    zero_division=0  
)  
pca_f1 = f1_score(  
    y_test,  
    pca_prediction,  
    zero_division=0  
)  
print(  
    "\n===== PCA + NAIVE BAYES ====="  
)  
print(  
    "Accuracy:",  
    round(pca_accuracy, 4)  
)  
print(  
    "Precision:",  
    round(pca_precision, 4)  
)  
print(  
    "Recall:",  
    round(pca_recall, 4)  
)
```

```
print(
    "F1 Score:",
    round(pca_f1, 4)
)
comparison = pd.DataFrame({
    "Model": [
        "Original Naive Bayes",
        "Feature Selection",
        "PCA + Naive Bayes"
    ],
    "Accuracy": [
        original_accuracy,
        optimized_accuracy,
        pca_accuracy
    ],
    "Precision": [
        original_precision,
        optimized_precision,
        pca_precision
    ],
    "Recall": [
        original_recall,
        optimized_recall,
        pca_recall
    ],
    "F1 Score": [
        original_f1,
        optimized_f1,
        pca_f1
    ]
})
print(
```

```

        "\n===== MODEL COMPARISON ====="
    )
    print(
        comparison.to_string(
            index=False
        )
    )

    print(
        "\n===== FINAL CLASSIFICATION REPORT ====="
    )

    print(
        classification_report(
            y_test_reduced,
            optimized_prediction,
            target_names=[
                "Benign",
                "Malignant"
            ],
            zero_division=0
        )
    )

```

## Output:

```

*** Choose Files data.csv
*** data.csv(text/csv) - 125204 bytes, last modified: 8/12/2026 - 100% done
Saving data.csv to data (1).csv

Dataset loaded successfully!
Dataset Shape: (569, 33)

===== MISSING VALUES =====
0

===== CORRELATION ANALYSIS =====

```

|                        | radius_mean | texture_mean | perimeter_mean | area_mean | \ |
|------------------------|-------------|--------------|----------------|-----------|---|
| radius_mean            | 1.00        | 0.32         | 1.00           | 0.99      |   |
| texture_mean           | 0.32        | 1.00         | 0.33           | 0.32      |   |
| perimeter_mean         | 1.00        | 0.33         | 1.00           | 0.99      |   |
| area_mean              | 0.99        | 0.32         | 0.99           | 1.00      |   |
| smoothness_mean        | 0.17        | -0.02        | 0.21           | 0.18      |   |
| compactness_mean       | 0.51        | 0.24         | 0.56           | 0.50      |   |
| concavity_mean         | 0.68        | 0.30         | 0.72           | 0.69      |   |
| concave points_mean    | 0.82        | 0.29         | 0.85           | 0.82      |   |
| symmetry_mean          | 0.15        | 0.07         | 0.18           | 0.15      |   |
| fractal_dimension_mean | -0.31       | -0.08        | -0.26          | -0.28     |   |
| radius_se              | 0.68        | 0.28         | 0.69           | 0.73      |   |
| texture_se             | -0.10       | 0.39         | -0.09          | -0.07     |   |
| perimeter_se           | 0.67        | 0.28         | 0.69           | 0.73      |   |
| area_se                | 0.74        | 0.26         | 0.74           | 0.80      |   |
| smoothness_se          | -0.22       | 0.01         | -0.20          | -0.17     |   |
| compactness_se         | 0.21        | 0.19         | 0.25           | 0.21      |   |
| concavity_se           | 0.19        | 0.14         | 0.23           | 0.21      |   |
| concave points_se      | 0.38        | 0.16         | 0.41           | 0.37      |   |
| symmetry_se            | -0.10       | 0.01         | -0.08          | -0.07     |   |
| fractal_dimension_se   | -0.04       | 0.05         | -0.01          | -0.02     |   |

```

... ===== ORIGINAL NAIVE BAYES =====
Accuracy: 0.9211
Precision: 0.9231
Recall: 0.8571
F1 Score: 0.8889

===== ORIGINAL CONFUSION MATRIX =====
[[69  3]
 [ 6 36]]

===== FEATURES REMOVED =====
['perimeter_mean', 'area_mean', 'concavity_mean', 'concave points_mean', 'perimeter_

Original number of features: 30
Reduced number of features: 13

===== OPTIMIZED NAIVE BAYES =====
Accuracy: 0.8684
Precision: 0.814
Recall: 0.8333
F1 Score: 0.8235

===== OPTIMIZED CONFUSION MATRIX =====
[[64  8]
 [ 7 35]]

===== PCA ANALYSIS =====
Original features: 30
PCA components: 10
Explained variance: 0.9521

```

```

===== MODEL COMPARISON =====
          Model  Accuracy  Precision  Recall  F1 Score
Original Naive Bayes  0.921053  0.923077  0.857143  0.888889
Feature Selection    0.868421  0.813953  0.833333  0.823529
PCA + Naive Bayes    0.894737  0.857143  0.857143  0.857143

===== FINAL CLASSIFICATION REPORT =====
          precision    recall  f1-score   support

   Benign           0.90      0.89      0.90        72
  Malignant         0.81      0.83      0.82        42

 accuracy                   0.87        114
  macro avg           0.86      0.86      0.86        114
 weighted avg         0.87      0.87      0.87        114

```

## Conclusion

The Naïve Bayes classifier can be affected when input features are highly correlated because its conditional independence assumption is violated. Correlated features may contain overlapping information, causing the classifier to count similar evidence multiple times and produce overconfident probability estimates.

The feature relationships were analyzed using correlation coefficients, and highly correlated features were identified. The model was then optimized using feature selection and PCA dimensionality reduction. The original and optimized models were compared using accuracy, precision, recall, F1-score, and confusion matrix to determine whether removing redundant information improves classification performance.

**Q9. A logistic regression model is unable to clearly separate two classes due to overlapping feature distributions.**

**(i) Identify the cause of overlapping decision boundaries.**

**(ii) Analyze the feature space and classification errors.**

**(iii) Improve the model using techniques such as feature transformation, polynomial features, or regularization.**

## **Logistic Regression – Overlapping Feature Distributions**

### **Problem Statement**

A **Logistic Regression** model is unable to clearly separate two classes because the feature values of the two classes overlap. When both classes have similar feature distributions, the model cannot create a clear decision boundary, resulting in incorrect classifications.

Logistic Regression estimates the probability of belonging to Class 1 as:

where:

The usual decision rule is:

Probability  $\geq 0.5 \rightarrow$  Class 1

Probability  $< 0.5 \rightarrow$  Class 0

### **(i) Identify the Cause of Overlapping Decision Boundaries**

#### **1. Overlapping Feature Distributions**

The two classes may have similar feature values.

For example:

Class 0  $\rightarrow$  40, 45, 50, 55

Class 1  $\rightarrow$  50, 55, 60, 65

The values **50–55 overlap**, making classification difficult.

#### **2. Linear Boundary Limitation**

Standard Logistic Regression creates a **linear decision boundary**. If the actual relationship between the features and classes is non-linear, a straight boundary cannot separate the classes properly.

#### **3. Irrelevant Features**

Unimportant or noisy features can make the model learn an ineffective boundary.

#### **4. Incorrect Feature Representation**

The original features may not adequately represent the relationship between the classes.

#### **5. Outliers**

Extreme observations can influence the regression coefficients and shift the decision boundary.

## 6. Insufficient Information

If the two classes genuinely have very similar characteristics, even a sophisticated model may have difficulty achieving perfect separation.

### (ii) Analyze the Feature Space and Classification Errors

#### 1. Examine Feature Relationships

Analyze each feature using:

- Mean
- Standard deviation
- Minimum
- Maximum
- Class-wise statistics

If the values for Class 0 and Class 1 are very similar, there is significant feature overlap.

#### 2. Analyze Classification Errors

A **confusion matrix** can be used:

|                | Predicted Class 0 | Predicted Class 1 |
|----------------|-------------------|-------------------|
| Actual Class 0 | TN                | FP                |
| Actual Class 1 | FN                | TP                |

##### False Positive

The model incorrectly assigns Class 1 to a Class 0 observation.

##### False Negative

The model fails to identify a Class 1 observation.

#### 3. Performance Metrics

Use:

**Accuracy**

**Precision**

**Recall**

**F1-score**

These metrics help determine whether errors are mainly false positives or false negatives.

### (iii) Improve the Model

#### 1. Feature Transformation

Transform features so that the relationship between the features and target becomes easier to model.

Examples:

- Log transformation

- Square-root transformation
- Standardization
- Normalization

### **Dataset:**

Kaggle – Breast Cancer Wisconsin Diagnostic Dataset

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

### **code:**

```
import pandas as pd
import numpy as n
from google.colab import files
from sklearn.model_selection import train_test_spli
from sklearn.preprocessing import (
    StandardScaler,
    PolynomialFeatures
)
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
import warnings
warnings.filterwarnings("ignore")
print("Upload data.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
```

```

print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
if "Unnamed: 32" in df.columns:
    df = df.drop(
        columns=["Unnamed: 32"]
    )
if "id" in df.columns:
    df = df.drop(
        columns=["id"]
    )
df["diagnosis"] = df[
    "diagnosis"
].map({
    "M": 1,
    "B": 0
})

"\n===== DATA SPLIT ====="
)
print(
    "Training samples:",
    len(X_train)
)
print(
    "Testing samples:",
    len(X_test)
)
baseline_model = Pipeline([
    (
        "scaler",
        StandardScaler()

```

```
),

(
    "logistic",
    LogisticRegression(
        penalty="l2",
        C=1.0,
        max_iter=5000
    )
)
])

baseline_model.fit(
    X_train,
    y_train
)

baseline_prediction = (
    baseline_model.predict(
        X_test
    )
)

baseline_accuracy = accuracy_score(
    y_test,
    baseline_prediction
)

baseline_precision = precision_score(
    y_test,
    baseline_prediction,
    zero_division=0
)

baseline_recall = recall_score(
    y_test,
    baseline_prediction,
```

```

        zero_division=0
    )
    baseline_f1 = f1_score(
        y_test,
        baseline_prediction,
        zero_division=0
    )
    print(
        "\n===== BASELINE LOGISTIC REGRESSION ====="
    )
    print(
        "Accuracy:",
        round(baseline_accuracy, 4)
    )

    print(
        "Precision:",
        round(baseline_precision, 4)
    )
    print(
        "Recall:",
        round(baseline_recall, 4)
    )
    print(
        "F1 Score:",
        round(baseline_f1, 4)
    )
    print(
        "\n===== BASELINE CONFUSION MATRIX ====="
    )
    print(
        confusion_matrix(

```

```

        y_test,
        baseline_prediction
    )
)polynomial_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    (
        "logistic",
        LogisticRegression(
            penalty="l2",
            C=0.1,
            max_iter=10000
        )
    )
])
polynomial_model.fit(
    X_train,
    y_train
)
polynomial_prediction = (
    polynomial_model.predict(
        X_test
    )
)
poly_accuracy = accuracy_score(
    y_test,
    polynomial_prediction
)
poly_precision = precision_score(
    y_test,
    polynomial_prediction,

```

```

    zero_division=0
)
poly_recall = recall_score(
    y_test,
    polynomial_prediction,
    zero_division=0
)
poly_f1 = f1_score(
    y_test,
    polynomial_prediction,
    zero_division=0
)
print(
    "\n===== POLYNOMIAL LOGISTIC REGRESSION
=====")
)
print(
    "Accuracy:",
    round(poly_accuracy, 4)
)
print(
    "Precision:",
    round(poly_precision, 4)
)
print(
    "Recall:",
    round(poly_recall, 4)
)
print(
    "F1 Score:",
    round(poly_f1, 4)
)

```

```
l1_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),
    (
        "logistic",
        LogisticRegression(
            penalty="l1",
            solver="liblinear",
            C=0.1,
            max_iter=5000
        )
    )
])
l1_model.fit(
    X_train,
    y_train
)
l1_prediction = l1_model.predict(
    X_test
)
l1_accuracy = accuracy_score(
    y_test,
    l1_prediction
)
l1_precision = precision_score(
    y_test,
    l1_prediction,
    zero_division=0
)
l1_recall = recall_score(
```

```
    y_test,
    l1_prediction,
    zero_division=0
)

l1_f1 = f1_score(
    y_test,
    l1_prediction,
    zero_division=0
)

print(
    "\n===== L1 REGULARIZATION ====="
)

print(
    "Accuracy:",
    round(l1_accuracy, 4)
)

print(
    "Precision:",
    round(l1_precision, 4)
)

print(
    "Recall:",
    round(l1_recall, 4)
)

print(
    "F1 Score:",
    round(l1_f1, 4)
)
```

```
l2_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),

    (
        "logistic",
        LogisticRegression(
            penalty="l2",
            C=0.1,
            max_iter=5000
        )
    )
])
l2_model.fit(
    X_train,
    y_train
)
```

```
l2_prediction = l2_model.predict(
    X_test
)
l2_accuracy = accuracy_score(
    y_test,
    l2_prediction
)
l2_precision = precision_score(
    y_test,
    l2_prediction,
    zero_division=0
)
```

```

)
l2_recall = recall_score(
    y_test,
    l2_prediction,
    zero_division=0
)
l2_f1 = f1_score(
    y_test,
    l2_prediction,
    zero_division=0
)
print(
    "\n===== L2 REGULARIZATION ====="
)
print(
    "Accuracy:",
    round(l2_accuracy, 4)

print(
    "Precision:",
    round(l2_precision, 4)
)
print(
    "Recall:",
    round(l2_recall, 4)
)
print(
    "F1 Score:",
    round(l2_f1, 4)
)
comparison = pd.DataFrame({
    "Model": [

```

```

        "Baseline Logistic Regression",
        "Polynomial Logistic Regression",
        "L1 Regularization",
        "L2 Regularization"
    ],
    "Accuracy": [
        baseline_accuracy,
        poly_accuracy,
        l1_accuracy,
        l2_accuracy
    ],
    "Precision": [
        baseline_precision,
        poly_precision,
        l1_precision,
        l2_precision
    ],
    "Recall": [
        baseline_recall,
        poly_recall,
        l1_recall,
        l2_recall
    ],
    "F1 Score": [
        baseline_f1,
        poly_f1,
        l1_f1,
        l2_f1
    ]
})

print(
    "\n===== MODEL COMPARISON ====="

```

```

)
print(
    comparison.to_string(
        index=False
    )
)
)
C_values = [
    0.001,
    0.01,
    0.1,
    1.0,
    10.0
]
tuning_results = []
for c in C_values:
    model = Pipeline([
        (
            "scaler",
            StandardScaler()
        ),

        (
            "logistic",
            LogisticRegression(
                penalty="l2",
                C=c,
                max_iter=5000
            )
        )
    ])

    model.fit(

```

```
    X_train,  
    y_train  
)
```

```
prediction = model.predict(  
    X_test  
)
```

```
accuracy = accuracy_score(  
    y_test,  
    prediction  
)
```

```
precision = precision_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
f1 = f1_score(  
    y_test,  
    prediction,  
    zero_division=0  
)
```

```
tuning_results.append(  
    c,  
    accuracy,  
    precision,  
    recall,  
    f1
```

```

    ])

tuning_df = pd.DataFrame(
    tuning_results,
    columns=[
        "C",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(
    "\n===== REGULARIZATION TUNING ====="
)

print(
    tuning_df.to_string(
        index=False
    )
)

best_index = tuning_df[
    "F1 Score"
].idxmax()

best_C = tuning_df.loc[
    best_index,
    "C"
]

best_accuracy = tuning_df.loc[
    best_index,
    "Accuracy"
]

```

```

best_precision = tuning_df.loc[
    best_index,
    "Precision"
]

best_recall = tuning_df.loc[
    best_index,
    "Recall"
]

best_f1 = tuning_df.loc[
    best_index,
    "F1 Score"
]

print(
    "\n===== BEST REGULARIZATION ====="
)

print(
    "Best C:",
    best_C

print(
    "Accuracy:",
    round(best_accuracy, 4)
)

print(
    "Precision:",
    round(best_precision, 4)
)

print(
    "Recall:",
    round(best_recall, 4)
)

```

```

print(
    "F1 Score:",
    round(best_f1, 4)
)
final_model = Pipeline([
    (
        "scaler",
        StandardScaler()
    ),

    (
        "logistic",
        LogisticRegression(
            penalty="l2",
            C=best_C,
            max_iter=5000
        )
    )
])
final_model.fit(
    X_train,
    y_train

final_prediction = final_model.predict(
    X_test
)
print(
    "\n===== FINAL CLASSIFICATION REPORT ====="
)
print(
    classification_report(
        y_test,

```

```

        final_prediction,
        target_names=[
            "Benign",
            "Malignant"
        ],
        zero_division=0
    )
)
print(
    "\n===== FINAL CONFUSION MATRIX ====="
)
print(
    confusion_matrix(
        y_test,
        final_prediction
    )
)

```

## Output:

```

Choose Files data.csv
data.csv(text/csv) - 125204 bytes, last modified: 8/12/2026 - 100% done
... Saving data.csv to data (2).csv

Dataset loaded successfully!
Dataset Shape: (569, 33)

===== MISSING VALUES =====
0

===== FEATURE SPACE ANALYSIS =====

```

|       | radius_mean | texture_mean | perimeter_mean | area_mean | smoothness_mean | \ |
|-------|-------------|--------------|----------------|-----------|-----------------|---|
| count | 569.00      | 569.00       | 569.00         | 569.00    | 569.00          |   |
| mean  | 14.13       | 19.29        | 91.97          | 654.89    | 0.10            |   |
| std   | 3.52        | 4.30         | 24.30          | 351.91    | 0.01            |   |
| min   | 6.98        | 9.71         | 43.79          | 143.50    | 0.05            |   |
| 25%   | 11.70       | 16.17        | 75.17          | 420.30    | 0.09            |   |
| 50%   | 13.37       | 18.84        | 86.24          | 551.10    | 0.10            |   |
| 75%   | 15.78       | 21.80        | 104.10         | 782.70    | 0.11            |   |
| max   | 28.11       | 39.28        | 188.50         | 2501.00   | 0.16            |   |

|       | compactness_mean | concavity_mean | concave | points_mean | symmetry_mean | \ |
|-------|------------------|----------------|---------|-------------|---------------|---|
| count | 569.00           | 569.00         |         | 569.00      | 569.00        |   |
| mean  | 0.10             | 0.09           |         | 0.05        | 0.18          |   |
| std   | 0.05             | 0.08           |         | 0.04        | 0.03          |   |
| min   | 0.02             | 0.00           |         | 0.00        | 0.11          |   |
| 25%   | 0.06             | 0.03           |         | 0.02        | 0.16          |   |
| 50%   | 0.09             | 0.06           |         | 0.03        | 0.18          |   |
| 75%   | 0.13             | 0.13           |         | 0.07        | 0.20          |   |
| max   | 0.35             | 0.43           |         | 0.20        | 0.30          |   |

```

***
===== CLASS-WISE FEATURE MEANS =====
      radius_mean  texture_mean  perimeter_mean  area_mean  \
diagnosis
0              12.15          17.91           78.08    462.79
1              17.46          21.60          115.37    978.38

      smoothness_mean  compactness_mean  concavity_mean  \
diagnosis
0              0.09          0.08          0.05
1              0.10          0.15          0.16

      concave points_mean  symmetry_mean  fractal_dimension_mean  ...  \
diagnosis
0              0.03          0.17          0.06  ...
1              0.09          0.19          0.06  ...

      radius_worst  texture_worst  perimeter_worst  area_worst  \
diagnosis
0              13.38          23.52           87.01    558.90
1              21.13          29.32          141.37   1422.29

      smoothness_worst  compactness_worst  concavity_worst  \
diagnosis
0              0.12          0.18          0.17
1              0.14          0.37          0.45

      concave points_worst  symmetry_worst  fractal_dimension_worst
diagnosis
0              0.07          0.27          0.08
1              0.18          0.32          0.09

```

[2 rows x 30 columns]

- ===== DATA SPLIT =====

Training samples: 455  
Testing samples: 114

===== BASELINE LOGISTIC REGRESSION =====

Accuracy: 0.9649  
Precision: 0.975  
Recall: 0.9286  
F1 Score: 0.9512

===== BASELINE CONFUSION MATRIX =====

```
[[71  1]
 [ 3 39]]
```

===== POLYNOMIAL LOGISTIC REGRESSION =====

Accuracy: 0.9386  
Precision: 0.9268  
Recall: 0.9048  
F1 Score: 0.9157

===== L1 REGULARIZATION =====

Accuracy: 0.9825  
Precision: 1.0  
Recall: 0.9524  
F1 Score: 0.9756

===== L2 REGULARIZATION =====

Accuracy: 0.9825  
Precision: 1.0  
Recall: 0.9524  
F1 Score: 0.9756

```

===== MODEL COMPARISON =====
...
      Model Accuracy Precision Recall F1 Score
Baseline Logistic Regression 0.964912 0.975000 0.928571 0.951220
Polynomial Logistic Regression 0.938596 0.926829 0.904762 0.915663
      L1 Regularization 0.982456 1.000000 0.952381 0.975610
      L2 Regularization 0.982456 1.000000 0.952381 0.975610

===== REGULARIZATION TUNING =====
      C Accuracy Precision Recall F1 Score
0.001 0.877193 1.00000 0.666667 0.800000
0.010 0.947368 1.00000 0.857143 0.923077
0.100 0.982456 1.00000 0.952381 0.975610
1.000 0.964912 0.97500 0.928571 0.951220
10.000 0.973684 0.97561 0.952381 0.963855

===== BEST REGULARIZATION =====
Best C: 0.1
Accuracy: 0.9825
Precision: 1.0
Recall: 0.9524
F1 Score: 0.9756

===== FINAL CLASSIFICATION REPORT =====
      precision recall f1-score support
Benign      0.97    1.00    0.99        72
Malignant   1.00    0.95    0.98        42

accuracy      0.98    114
macro avg     0.99    0.98    0.98    114
weighted avg  0.98    0.98    0.98    114

===== FINAL CONFUSION MATRIX =====
[[72  0]
 [ 2 40]]

```

## Conclusion

The Logistic Regression model had difficulty separating the two classes because their feature distributions overlapped, while a standard Logistic Regression model uses a linear decision boundary. The feature space and classification errors were analyzed using class-wise statistics, confusion matrix, accuracy, precision, recall, and F1-score.

The model was improved using feature scaling, polynomial feature transformation, and L1/L2 regularization. Different regularization strengths were tested, and the best value was selected based on F1-score. These techniques provide a more flexible and stable model for handling overlapping class distributions and can improve classification performance.

**Q10. A discriminant analysis model is too simple and fails to capture the complexity of the dataset, resulting in low accuracy.**

**(i) Identify the signs of underfitting in the model.**

**(ii) Analyze model bias and feature limitations.**

**(iii) Apply optimization strategies such as adding features, increasing model complexity, or using nonlinear transformations.**

### **Problem Statement**

A **discriminant analysis model** is too simple to capture the complexity of the dataset, resulting in low classification accuracy. This problem is called **underfitting**.

Underfitting occurs when a model is unable to learn the important patterns and relationships present in the training data.

#### **(i) Signs of Underfitting**

##### **1. Low Training Accuracy**

If the model performs poorly even on the training dataset, it has failed to learn the underlying patterns.

Example:

Training Accuracy = 68%

Testing Accuracy = 66%

Both values are low, indicating underfitting.

##### **2. High Training and Testing Error**

An underfitted model produces relatively high errors on both training and testing data.

##### **3. Small Training–Testing Gap**

Unlike overfitting, there may be only a small difference between training and testing performance.

Training Accuracy = 70%

Testing Accuracy = 68%

The problem is not memorization; the model is simply too simple.

##### **4. Poor F1-Score**

Low precision, recall, and F1-score indicate that the model is not learning useful class-discriminating patterns.

##### **5. Oversimplified Decision Boundary**

A simple linear discriminant may be unable to represent complex relationships between features.

## **(ii) Analyze Model Bias and Feature Limitations**

### **1. High Bias**

Underfitting is generally associated with **high bias**.

The model makes strong simplifying assumptions and cannot represent the actual relationship between the features and classes.

### **2. Limited Features**

The model may not have enough useful features to distinguish between the classes.

For example:

Original:

Feature 1

Feature 2

But the target may depend on:

Feature 1

Feature 2

Feature 3

Feature 4

Feature interactions

Adding relevant features can provide more information.

### **3. Linear Model Limitation**

If the actual class relationship is nonlinear, a simple linear discriminant may produce low accuracy.

### **4. Feature Interactions**

Two features may individually provide limited information but become useful when considered together.

For example:

can represent an interaction between two variables.

## **(iii) Optimization Strategies**

### **1. Add Relevant Features**

Include additional meaningful variables that can improve class separation.

Examples:

- Interaction features
- Statistical features
- Domain-specific variables

- Aggregated features

## 2. Increase Model Complexity

Instead of using only a simple linear discriminant, a more flexible model can be used.

For example:

- Quadratic Discriminant Analysis (QDA)
- Polynomial feature transformation
- Nonlinear feature mapping

QDA allows each class to have a different covariance structure and therefore provides a more flexible decision boundary than Linear Discriminant Analysis.

## 3. Nonlinear Feature Transformation

Polynomial features can create additional representations such as:

This allows the classifier to capture nonlinear relationships.

## 4. Feature Scaling

Standardization can ensure that features with large numerical ranges do not dominate the model.

### Dataset:

Kaggle – Breast Cancer Wisconsin Diagnostic Dataset

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

### code:

```
import pandas as pd
import numpy as np
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```

        fl_score,
        confusion_matrix,
        classification_report
    )
import warnings
warnings.filterwarnings("ignore")
print("Upload data.csv")
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("\nDataset loaded successfully!")
print("Dataset Shape:", df.shape)
if "Unnamed: 32" in df.columns
    df = df.drop(
        columns=["Unnamed: 32"]
    )
if "id" in df.columns:
    df = df.drop(
        columns=["id"]
    )
df["diagnosis"] = df[
    "diagnosis"
].map({
    "M": 1,
    "B": 0
})
X_test_scaled = scaler.transform(
    X_test
)
lda_model = LinearDiscriminantAnalysis()

lda_model.fit(

```

```
    X_train_scaled,
    y_train
)
lda_train_prediction = (
    lda_model.predict(
        X_train_scaled
    )
)
lda_test_prediction = (
    lda_model.predict(
        X_test_scaled
    )
)
lda_train_accuracy = accuracy_score(
    y_train,
    lda_train_prediction

lda_test_accuracy = accuracy_score(
    y_test,
    lda_test_prediction
)

lda_precision = precision_score(
    y_test,
    lda_test_prediction,
    zero_division=0

lda_recall = recall_score(
    y_test,
    lda_test_prediction,
    zero_division=0
```

```
lda_f1 = f1_score(
    y_test,
    lda_test_prediction,
    zero_division=0

print(
    "\n===== LDA MODEL ====="
)
print(
    "Training Accuracy:",
    round(lda_train_accuracy, 4)
)
print(
    "Testing Accuracy:",
    round(lda_test_accuracy, 4)
)
print(
    "Precision:",
    round(lda_precision, 4)
)

print(
    "Recall:",
    round(lda_recall, 4)
)
print(
    "F1 Score:",
    round(lda_f1, 4)

print(
    "\n===== LDA CONFUSION MATRIX ====="
)
```

```

print(
    confusion_matrix(
        y_test,
        lda_test_prediction
    )
)

selected_features = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean"
]

selected_features = [
    feature
    for feature in selected_features
    if feature in X.columns
]

X_nonlinear = X[
    selected_features
].copy()

```

```

for feature in selected_features:
    X_nonlinear[
        feature + "_square"
    ] = (
        X_nonlinear[feature] ** 2
    )

if len(selected_features) >= 2:
    X_nonlinear[
        "feature_interaction_1"
    ] = (
        X_nonlinear[feature1] * X_nonlinear[feature2]
    )

```

```

    ] = (
        X_nonlinear[
            selected_features[0]
        ]
        *
        X_nonlinear[
            selected_features[1]
        ]
    )
if len(selected_features) >= 4:
    X_nonlinear[
        "feature_interaction_2"
    ] = (
        X_nonlinear[
            selected_features[2]
        ]
        *
        X_nonlinear[
            selected_features[3]
        ]
    )

print(
    "\n===== FEATURE ENGINEERING ====="
)
print(
    "Original feature count:",
    X.shape[1]
)
print(
    "New feature count:",

```

```
    X_nonlinear.shape[1]
)_train_nl, X_test_nl, y_train_nl, y_test_nl = train_test_split(
    X_nonlinear,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
nl_scaler = StandardScaler(
X_train_nl_scaled = (
    nl_scaler.fit_transform(
        X_train_nl
    )
)
qda_model = QuadraticDiscriminantAnalysis(
    reg_param=0.1
)
qda_model.fit(
    X_train_nl_scaled,
    y_train_nl
)
qda_train_prediction = (
    qda_model.predict(
        X_train_nl_scaled
    )
)
qda_test_prediction = (
    qda_model.predict(
        X_test_nl_scaled
    )
)
qda_train_accuracy = accuracy_score(
    y_train_nl,
```

```

    qda_train_prediction
)
qda_test_accuracy = accuracy_score(
    y_test_nl,
    qda_test_prediction
)
print(
    "\n===== QDA OPTIMIZED MODEL ====="
)
print(
    "Training Accuracy:",
    round(qda_train_accuracy, 4)
)
print(
    "Testing Accuracy:",
    round(qda_test_accuracy, 4)
)
print(
    "Precision:",
    round(qda_precision, 4)
)

print(
    "Recall:",
    round(qda_recall, 4)
)
print(
    "F1 Score:",
    round(qda_f1, 4)
)

print(
    "\n===== QDA CONFUSION MATRIX ====="

```

```
)  
print(  
    confusion_matrix(  
        y_test_nl,  
        qda_test_prediction  
    )  
)  
comparison = pd.DataFrame({  
  
    "Model": [  
        "LDA - Simple",  
        "QDA - Optimized"  
    ],  
    "Training Accuracy": [  
        lda_train_accuracy,  
        qda_train_accuracy  
    ],  
    "Testing Accuracy": [  
        lda_test_accuracy,  
        qda_test_accuracy  
    ],  
    "Precision": [  
        lda_precision,  
        qda_precision  
    ],  
    "Recall": [  
        lda_recall,  
        qda_recall  
    ],  
  
    "F1 Score": [  
        lda_f1,
```

```

        qda_fl
    ]
})
print(
    "\n===== MODEL COMPARISON ====="
)
print(
    comparison.to_string(
        index=False
    )

reg_values = [
    0.0,
    0.01,
    0.05,
    0.1,
    0.2,
    0.5
]

tuning_results = []
for reg in reg_values:
    model = QuadraticDiscriminantAnalysis(
        reg_param=reg
    )
    model.fit(
        X_train_nl_scaled,
        y_train_nl
    )

    prediction = model.predict(
        X_test_nl_scaled

```

```

    )
    accuracy = accuracy_score(
        y_test_nl,
        prediction
    )
    precision = precision_score(
        y_test_nl,
        prediction,
        zero_division=0
    )
    recall = recall_score(
        y_test_nl,
        prediction,
        zero_division=0
    )
    f1 = f1_score(
        y_test_nl,
        prediction,
        zero_division=0
    )
    tuning_results.append([
        reg,
        accuracy,
        precision,
        recall,
        f1
    ])
tuning_df = pd.DataFrame(
    tuning_results,
    columns=[
        "Regularization",
        "Accuracy",

```

```

        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(
    "\n===== QDA REGULARIZATION TUNING ====="
)

print(
    tuning_df.to_string(
        index=False
    )
)

best_index = tuning_df[
    "F1 Score"
].idxmax()

best_reg = tuning_df.loc[
    best_index,
    "Regularization"

best_accuracy = tuning_df.loc[
    best_index,
    "Accuracy"

best_precision = tuning_df.loc[
    best_index,
    "Precision"
]

best_recall = tuning_df.loc[

```

```

        best_index,
        "Recall"
    ]
    best_f1 = tuning_df.loc[
        best_index,
        "F1 Score"
    ]
    print(
        "\n===== BEST QDA MODEL ====="
    )
    print(
        "Best Regularization:",
        best_reg
    )
    print(
        "Accuracy:",
        round(best_accuracy, 4)
    )
    print(
        "Precision:",
        round(best_precision, 4)
    )
    print(
        "Recall:",
        round(best_recall, 4)
    )
    print(
        "F1 Score:",
        round(best_f1, 4)
    )
    final_qda = QuadraticDiscriminantAnalysis(
        reg_param=best_reg

```

```

)
final_qda.fit(
    X_train_nl_scaled,
    y_train_nl

final_prediction = final_qda.predict(
    X_test_nl_scaled
)
print(
    "\n===== FINAL CLASSIFICATION REPORT ====="
)
print(
    classification_report(
        y_test_nl,
        final_prediction,
        target_names=[
            "Benign",
            "Malignant"
        ],
        zero_division=0
    )
)
print(
    "\n===== FINAL CONFUSION MATRIX ====="
)

print(
    confusion_matrix(
        y_test_nl,
        final_prediction
    )
)

```

## Output:

```
*** Upload data.csv
Choose Files data.csv
data.csv(text/csv) - 125204 bytes, last modified: 8/12/2026 - 100% done
Saving data.csv to data (3).csv

Dataset loaded successfully!
Dataset Shape: (569, 33)

===== MISSING VALUES =====
0

===== DATA SPLIT =====
Training samples: 455
Testing samples: 114

===== LDA MODEL =====
Training Accuracy: 0.9626
Testing Accuracy: 0.9649
Precision: 1.0
Recall: 0.9048
F1 Score: 0.95

===== LDA CONFUSION MATRIX =====
[[72  0]
 [ 4 38]]

===== FEATURE ENGINEERING =====
Original feature count: 30
New feature count: 12

*** Training Accuracy: 0.9275
Testing Accuracy: 0.8772
Precision: 0.8889
Recall: 0.7619
F1 Score: 0.8205

===== QDA CONFUSION MATRIX =====
[[68  4]
 [10 32]]

===== MODEL COMPARISON =====
      Model  Training Accuracy  Testing Accuracy  Precision  Recall  F1 Score
LDA - Simple      0.962637      0.964912      1.000000  0.904762  0.950000
QDA - Optimized    0.927473      0.877193      0.888889  0.761905  0.820513

===== QDA REGULARIZATION TUNING =====
Regularization  Accuracy  Precision  Recall  F1 Score
0.00  0.885965  0.939394  0.738095  0.826667
0.01  0.885965  1.000000  0.690476  0.816901
0.05  0.885965  0.914286  0.761905  0.831169
0.10  0.877193  0.888889  0.761905  0.820513
0.20  0.877193  0.888889  0.761905  0.820513
0.50  0.877193  0.888889  0.761905  0.820513

===== BEST QDA MODEL =====
Best Regularization: 0.05
Accuracy: 0.886
Precision: 0.9143
Recall: 0.7619
F1 Score: 0.8312

===== FINAL CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

   Benign           0.87       0.96       0.91        72
  Malignant         0.91       0.76       0.83        42

 accuracy                   0.89        114
  macro avg           0.89       0.86       0.87        114
  weighted avg        0.89       0.89       0.88        114

===== FINAL CONFUSION MATRIX =====
[[69  3]
 [10 32]]
```

## **Conclusion**

The discriminant analysis model was underfitting because its simple decision boundary and limited feature representation were unable to capture the complexity of the dataset. Underfitting is indicated by relatively low training and testing performance with a small performance gap.

The model was improved by adding relevant nonlinear features, creating interaction terms, increasing model flexibility from LDA to QDA, scaling the features, and tuning QDA regularization. The original and optimized models were compared using accuracy, precision, recall, F1-score, and the confusion matrix.